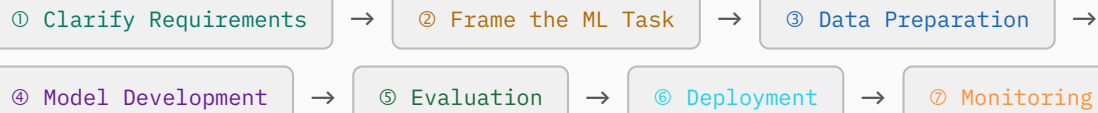


ML System Design · Master Quick Map

Framework → Systems fingerprint → Model choice (cold start) → Serving funnel · revision-first

The 7-Step Framework



Interview tip: Spend 3–4 min on Step 1. Skipping it is an immediate red flag. Everything flows from the business objective.

10 Systems · Architecture Fingerprint

SYSTEM	ML OBJECTIVE	CANDIDATE GEN	RANKING MODEL	KEY OFFLINE METRIC	PRIMARY ONLINE METRIC	RETRAIN FREQ
Visual Search (Pinterest)	Rank by visual similarity	ANN on image embeddings	Contrastive (SimCLR)	nDCG (graded 0–5)	CTR, daily time spent	Monthly
Street View Blurring (Google)	Detect faces & plates	Object detector regions	CNN classifier per region	mAP (IOU threshold)	User reports & complaints	Monthly
YouTube Video Search	Rank videos by text query	ANN on text+video embeds	Two-tower (text+video)	MRR (one ground truth)	Completion rate, watch time	Daily
Harmful Content (Meta)	Classify harm type	Triggered by flags/upload	Multi-task DNN (early fusion)	PR-AUC (imbalanced)	Prevalence, harmful impressions	Daily
Video Recommendation (YouTube)	Maximize watch time	MF + Two-tower ANN	Two-tower with content feats	nDCG	CTR, total watch time	Daily
Event Recommendation (Eventbrite)	Maximize registrations	Geo + category filters	Pointwise LTR (DNN)	mAP	CTR, conversion rate	Daily
Ad Click Prediction	Maximize CTR / revenue	Ad targeting rules	DeepFM	ROC-AUC, Log-loss	CTR, revenue lift, hide rate	Continuous
Similar Listings (Airbnb)	Predict next listing clicked	ANN on listing embeddings	Session-based (W2V style)	Avg rank of booked listing	CTR, session book rate	Weekly
				nDCG		Daily

SYSTEM	ML OBJECTIVE	CANDIDATE GEN	RANKING MODEL	KEY OFFLINE METRIC	PRIMARY ONLINE METRIC	RETRAIN FREQ
News Feed (Facebook)	Maximize engagement score	Friends + interest filters	Multi-task DNN (5 tasks)		CTR, reactions, time spent	
PYMK (LinkedIn)	Maximize connections formed	FoF + shared attributes	GNN / Two-tower scoring	mAP	# requests sent & accepted	Weekly

Cold Start & Model Choice Quick Guide

SITUATION	CHOICE
Rich interaction history, no new users	Matrix Factorization
New users / items (cold start)	Two-Tower NN
Short-term intent > long-term	Session-based (W2V)
Social graph data available	GNN
Sparse categorical features, ad click	DeepFM
Multiple correlated tasks, data-sparse	Multi-task DNN
Image+text may be jointly harmful	Early fusion
Each modality independently sufficient	Late fusion

Universal 3-Stage Serving Funnel

Candidate Generation billions → thousands

Lightweight model · ANN retrieval · rule-based filters · high recall, low precision



Ranking / Scoring thousands → hundreds

Heavy model · rich features · DeepFM / Two-tower · accuracy over speed



Re-ranking hundreds → top-k served

Business rules · diversity · freshness · clickbait demotion · regional restrictions

Step 1 · Clarifying Requirements

Most undervalued step – skipping it is an immediate red flag in any FAANG loop

7 Dimensions to Always Cover

BFD · CSP · P | "Before Finalizing Designs, Check Scale, Performance, Privacy" |

BFD = What are we building for? · CSP = How will it operate? · P = Should we / legally can we?

DIMENSION	QUESTIONS TO ASK	WHY IT MATTERS → IF SKIPPED
Business Objective	What KPI does this move — revenue, engagement, or safety? ► Is there a conflict between two KPIs? (e.g., CTR vs. revenue, engagement vs. harm)	Sets the North Star metric + loss function direction → If skipped: CTR model maximises clicks, not revenue (Ch8 Ad Click trap)
🔗 Metric Chain (common confusion point): Business Objective → North Star (what company tracks, never trained on) → Proxy Metric (what model trains on) → Loss Function (math form of proxy) North Star constrains which proxy is valid. Proxy directly determines loss function type. X Trap: The proxy sets the loss — not the business objective directly. CTR (binary) → BCE · Watch time (float) → Huber · Harm label (multi-label) → Weighted BCE per head. North Star (revenue, retention) is never directly trained on.		
Feature Support	Can users like/dislike/report/hide? Is there a social graph? ► Are interactions explicit or implicit? Positive-only or bidirectional?	Defines training signals + label source → If skipped: GNN proposed with no graph data; no likes = no natural label (Ch11 PYMK)
Data	How large? Labeled — and if so, how + what labeling window? ► Is there severe class imbalance? User-generated or system-logged?	Determines labeling strategy + loss function → If skipped: assume supervised on unlabeled data; wrong model family (Ch5: 500M posts, only 10K hand-labels)
Constraints	Cloud or on-device? Auto-improving? ► Acceptable FN vs. FP rate — is one legally worse? Human review queue capacity?	Drives model complexity limits + threshold policy → If skipped: GBDT proposed where continual learning required; FN rate never set (Ch3 Street View: Recall > Precision = legal hard constraint)
Scale	How many DAU? How large is the item catalog (thousands vs. billions)? ► Expected QPS? Embedding index size feasible in RAM?	Determines serving architecture + ANN strategy → If skipped: single-node at 58K QPS; two-stage retrieval never justified (Ch6 Video Rec)
Performance	Real-time or batch? What is the p99 SLA (not average)? ► Session-bounded or long-term history? Latency sub-budget per pipeline stage?	Drives batch vs. online + latency sub-budget decomposition → If skipped: heavy ranker at retrieval stage; p99 blown; session vs. long-term flipped (Ch9 Similar Listings)
Privacy & Ethics	PII / sensitive data? Anonymization needed? GDPR? ► Graph traversal depth permitted? Advertiser or demographic fairness constraints?	Affects feature inclusion + graph depth + fairness → If skipped: 3rd-degree FoF traversal = GDPR violation; demographic ad features = fairness violation (Ch11, Ch8)

Interview Ritual → Open with: "Before I design anything, let me run BFD·CSP·P — Business Objective, Feedback signals, Data, Constraints, Scale, Performance SLA, and Privacy. Spend 3–4 min here. Everything downstream depends on these answers."

North Star · Guard · Proxy Metric Framework

Staff Critical · NEW

Metric Type	Definition	Example (Video Rec)	Failure if Ignored
North Star Metric	The single most important long-term business metric. This is what the company ultimately optimizes for.	7-day retention, total watch time per week	Optimizing CTR instead leads to clickbait; long-term retention drops despite short-term gains
Guard Metrics	Metrics that must NOT degrade , even if north star improves. Defines acceptable trade-off space.	Content diversity, p99 latency <200ms, fairness across demographics	Model boosts watch time by recommending only viral content; diversity collapses; users churn
Proxy Metric	What the model actually optimizes for. Should correlate with north star, but the gap is a trap .	CTR, watch completion rate, likes	CTR up 15% but 6-month retention drops — clickbait optimized, satisfaction destroyed

Metric Hierarchy Rule: Always define all three levels before choosing an ML objective. Proxy metric must correlate with north star AND must be paired with guard metrics to prevent Goodhart's Law: "When a measure becomes a target, it ceases to be a good measure."

FAANG Probe: "You optimized for CTR and it went up 15%. But 6-month retention dropped. Why?" → CTR is a proxy metric. Maximizing clicks without quality leads to clickbait consumption → long-term churn. North star = long-term retention. CTR needed a quality co-signal (watch completion) as guard metric.

How to Define Guard Metrics — 3-Source Framework

Staff Critical · NEW

Core Question to ask for every system: "What can go wrong when I aggressively maximize my proxy metric?"
Guard metrics are the answer — derived from exactly 3 independent sources below.

SOURCE	ASK YOURSELF	GUARD METRIC EXAMPLES	FAILURE IF MISSED
① UX Collapse	"If my model over-optimizes the proxy, which user behavior breaks?"	Content diversity score; skip rate; dwell time; % repeat items in top-K; session bounce rate	Video Rec over-optimizes CTR → filter bubble → long-tail vanishes → users churn despite short-term gain
② Infra / System Constraints	"What hard technical limits must never be violated regardless of model quality?"	p99 latency < SLA (always!); human reviewer queue depth < capacity; GPU memory budget	Harmful Content: threshold too low floods review queue — ops breaks even if model recall improves
③ Fairness / Legal Hard Walls	"Are there demographic groups, legal requirements, or business stakeholders that must not be harmed?"	FNR per demographic group; min impression share per advertiser; graph traversal depth limit (GDPR)	Street View: global mAP is high, but miss rate on dark skin tones → GDPR liability; legal hard wall

Interview ritual: State the guard metric and the failure scenario it prevents in the same sentence. Example: "I'd track content diversity as a guard metric — because aggressively maximizing CTR without it collapses recommendations into a filter bubble, which destroys long-term retention even while short-term engagement looks healthy."

FAANG Probe: "What guard metrics would you set for an ETA prediction system?" → **UX Guard:** Systematic prediction bias (always under-predicting → user trust erodes) · **Infra Guard:** p99 inference latency <50ms at order placement · **Fairness Guard:** Equal MAE across delivery zones — don't under-serve rural pin codes.

RSMA: 4-Pillar System Requirements

Chip Huyen · Staff+

PILLAR	WHAT TO SAY IN INTERVIEW	SILENT RISK IF IGNORED
Reliability	The system must perform correctly even under hardware/software faults. Unlike traditional software, ML fails silently — wrong predictions produce no crash or 404 error.	Model returns confidently wrong answers with no alert firing. Revenue drops; no engineer is paged.
Scalability	We grow in 3 dimensions: model complexity , traffic volume , and model count . Artifact management (versioning, rollbacks) is as critical as resource auto-scaling.	Monolithic model pipeline collapses under load or versioning chaos causes irreproducible models.
Maintainability	Code, data, and model artifacts must be versioned. ML engineers, DevOps, and domain SMEs must collaborate — tooling must accommodate all stakeholders.	Unreproducible experiments, broken rollbacks, and untracked feature schema changes corrupt models.
Adaptability	Data distributions shift continuously. The system must support updates without service interruption and must automatically discover performance regressions before users report them.	Model silently degrades as world changes (seasonality, concept drift). No retraining trigger fires.

RELIABILITY – Say This In Interview

"Before I design anything — ML fails silently. Wrong predictions produce no crash or alert. I need **online KPIs tied to business outcomes**, not just offline accuracy."

↳ **Ad Click (Ch8)**: CTR model confident at 80% → revenue drops post-deploy, no alert | **Harmful (Ch5)**: harm score 0.49, post stays live, goes viral | **Video Rec (Ch6)**: CTR ↑ 15% but 6-month retention ↓ = silent north-star divergence

SCALABILITY – 3 Dimensions (Not Just Traffic)

DIMENSION	DRIVER / FORMULA	ARCHITECTURE CONSEQUENCE
Traffic Volume	$\text{QPS} = \text{DAU} \times \text{req/day} \div 86,400$	$\text{QPS} > 3\text{--}5\text{K} \rightarrow$ Distributed inference fleet (Video Rec: 58K QPS)
Model Complexity	$\text{Index} = \text{items} \times \text{dims} \times 4\text{B (float32)}$	$\text{Index} > \text{RAM} \rightarrow$ Faiss IVF / HNSW sharding (YT Search: 5.12 TB)
Model Count	Artifact lineage, versioning, rollbacks	No Model Store → versioning chaos, irreproducible models (Ad Click continual)

MAINTAINABILITY – 3 Failure Modes to Name Proactively

- ↳ **Untracked feature schema changes** — upstream pipeline changes, model silently gets wrong inputs → **train-serve skew** (Event Rec: Maps API update corrupts distance feature)
- ↳ **Broken rollbacks** — no Model Store, can't revert to last known good → **outage with no escape hatch** (Ad Click: bad checkpoint, hourly retrain pipeline)
- ↳ **Untracked experiments** — two engineers, same config, different results → **irreproducible models** (PYMK: new graph edge type silently changes feature schema)

Fix stack: **Feature Store** (shared transform code) + **Model Store** (8-artifact checklist) + **DVC** (dataset versioning)

☆ **Staff+ Badge**: Raise RSMA proactively before the interviewer asks. Jumping straight to scale without mentioning silent failures or adaptability signals mid-level thinking.

FAANG Probe: "Your model has 95% accuracy offline but revenue dropped post-deployment. What happened?" → **Silent ML failure**. Offline metric diverged from business KPI. Likely: distribution shift, or ML objective (CTR) misaligned with business goal (revenue). Fix: add online A/B metrics tied to business KPIs, not just ML accuracy.

Capacity Estimation Template

Staff Non-Negotiable · NEW

DIMENSION	FORMULA	EXAMPLE (VIDEO REC, 1B DAU)	ARCHITECTURE GATE
QPS (queries/sec)	$\text{DAU} \times \text{req/day} \div 86,400$	$1\text{B} \times 5 \div 86,400 \approx 58\text{K QPS}$	Cannot use single-node serving; need distributed inference fleet
Embedding Index Size	$\text{Items} \times \text{dims} \times 4 \text{ bytes (float32)}$	$1\text{B items} \times 128\text{-dim} \approx 512 \text{ GB}$	Cannot load into RAM; need Faiss IVF + HNSW sharding across nodes
Latency SLA	State p50 AND p99 — never "average"	p50 <50ms, p99 <200ms	p99 drives architecture: pre-compute vs. real-time ANN retrieval
Compute Budget	$\text{retrain_freq} \times \text{data_rows} \times \text{model_size}$	Daily retrain on 100M rows with DNN	Cannot use batch-only GBDT; need incremental NN fine-tuning
Pre-compute Feasibility	$\text{Users} \times \text{Items} = \text{total pairs}$	$500\text{M} \times 50\text{M} = 25 \times 10^{15}$ — infeasible	Must use ANN retrieval with pre-computed item embeddings + real-time user embeddings

☆ **Staff+ Badge:** Capacity estimation gates architecture choices. You cannot defend a real-time two-tower without knowing your QPS. You cannot propose Faiss without knowing your embedding index size. Always estimate before proposing architecture.

Capacity Estimation · Deep-Dive Explainers

Session Notes · NEW

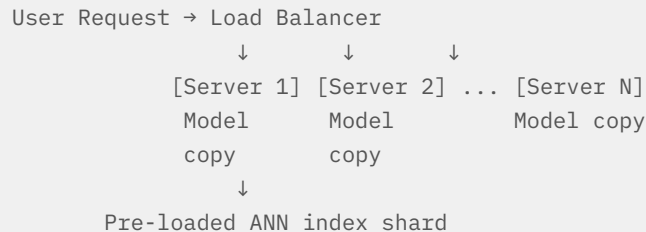
① QPS → Distributed Inference Fleet

A **distributed inference fleet** is a pool of servers each running a copy of the model, with a load balancer routing requests across them. Single-node serving becomes the bottleneck once QPS exceeds ~3–5K requests/sec per node.

Video Rec example (1B DAU):

$$\text{QPS} = 1\text{B} \times 5 \text{ req/day} \div 86,400 \approx 58\text{K QPS}$$

$$\text{Servers} = 58\text{K QPS} \div 3\text{K QPS/server} \approx 20 \text{ nodes (+ 2x headroom for spikes)}$$



Interview move: State the math → declare "this exceeds single-node capacity" → propose the fleet. Don't jump to fleet without the QPS justification.

② Index Size → Faiss IVF / HNSW Sharding

QPS drives how many nodes. Index size drives how those nodes store and query embeddings. These are independent dimensions that combine in serving design.

ALGORITHM	HOW IT WORKS	TRADE-OFF
Faiss IVF (Inverted File)	Clusters items into N buckets; searches only nearest buckets at query time	Faster; can miss items at cluster boundaries
HNSW (Hierarchical Navigable Small World)	Multi-layer graph; traverses from coarse to fine layers	Very fast query (~1ms); high memory overhead

Why sharding?

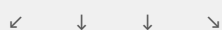
$$10\text{B videos} \times 128\text{-dim} \times 4 \text{ bytes} = 5.12 \text{ TB} \rightarrow \text{no single node has this RAM}$$

Shard across 20 nodes → each holds ~256 GB of the index

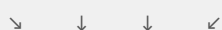
User Embedding Query



ANN Coordinator (fan-out)



Shard1 Shard2 Shard3 Shard4 ← each holds 1/4 of index



Merge → Top-K returned

Item embeddings: batch pre-computed OFFLINE, loaded into shards

User embeddings: computed REAL-TIME at query time (user tower)

③ Latency SLA → p99 Sub-Budget Decomposition

The p99 SLA is a **total budget**. Distribute it across every pipeline hop. The cheatsheet gives explicit targets for Stage 1 (<20ms) and Stage 2 (<100ms). Remaining budget goes to network + user embedding + re-ranking.

PIPELINE STAGE	BUDGET (P99 = 200MS)	SOURCE
User Embedding Inference	~10–20ms	User tower forward pass
ANN Retrieval (Stage 1)	<20ms	Cheatsheet explicit target
Heavy Ranker (Stage 2)	<100ms	Cheatsheet explicit target
Network RTT (Load Balancer)	~20–30ms	~10ms per hop, back-of-envelope
Re-ranking (business rules)	~10ms	No model — pure scoring logic

Key insight: ANN must be <20ms specifically to protect the ranker's budget. If ANN takes 80ms, the ranker only gets 60ms — not enough for a 200-feature DNN over 2K candidates. The constraint flows from p99 inward, not the other way.

Interview move: "My total p99 budget is 200ms. Stage 1 ANN must be under 20ms to leave budget for my Stage 2 ranker which needs ~100ms. Network overhead consumes ~30ms. That leaves ~50ms for user embedding and re-ranking."

④ Compute Budget → Feasibility Trigger, Not a Threshold

The formula $\text{retrain_freq} \times \text{data_rows} \times \text{model_size}$ does **not** produce a number you compare against a fixed threshold. Its output is a **feasibility question**: "Can a full retrain from scratch complete within the retraining window?"

SYSTEM	DATA / CADENCE	FULL RETRAIN FEASIBLE?	FORCED CHOICE
Ad Click	500M rows, continuous	No — takes hours; 5-min staleness deadline violated	Stateful fine-tuning on delta data → NN only (GBDT eliminated)
Video Rec	100M rows, daily	Borderline — feasible with large GPU cluster	Stateful Stage 3 preferred for efficiency
PYMK	~10M rows, weekly	Yes — comfortably fits a weekend run	Stateless OK → GBDT or GNN both valid

Decision logic (not a formula output):

Step 1: Estimate one full retrain cost → $\text{data_rows} \times \text{model_size}$ → "how big?"
Step 2: Compare to retrain window → "does it fit?"
Step 3:
YES → Stateless feasible → GBDT or NN both valid
NO → Must go Stateful (fine-tune on delta) → GBDT eliminated → NN required

Feasibility = retrain duration vs. staleness window (not a fixed threshold)

The staleness window is system-specific and comes from business stakes, not a formula output:

- Ad Click: bid cycle = 5 min → even a 1-hr retrain violates the window → **Stateful mandatory**
- Video Rec: daily refresh acceptable → 6–8 hr retrain borderline → **Stateful preferred**
- PYMK: weekly cadence → any reasonable retrain fits → **Stateless OK**

The staleness window kills feasibility, not raw GPU time alone. Always ask: "How stale can my model be before it costs money or safety?"

Why GBDT gets eliminated: GBDT cannot be incrementally fine-tuned on new data. It must retrain from scratch. For high-frequency retraining (daily/continuous), this is computationally infeasible at FAANG scale. Neural Networks support checkpoint-based stateful training on delta data only.

Stakeholder Conflict & Decoupled Objectives Pattern

Staff Trap · NEW

Core Rule (Chip Huyen): Never combine conflicting objectives into a single loss function. Optimizing for engagement alone inadvertently promotes extreme/clickbait/harmful content.

CONFLICT TYPE	EXAMPLE	X WRONG APPROACH	✓ CORRECT PATTERN
Engagement vs. Safety	CTR ranker amplifies misinformation	Penalize harm inside CTR loss — couples objectives, forces retraining to adjust policy	Train separate harm classifier; inject as guard signal at serving time
Revenue vs. User Trust	Ad CTR vs. hide-rate conflict	Single model optimizing revenue ignores user satisfaction signals	Separate CTR model + hide-rate predictor; combine at serving
CTR vs. Quality	Clickbait scores high CTR, low completion rate	Optimize CTR only; no quality co-signal	Final Score = $w_1 \cdot \text{CTR} + w_2 \cdot \text{completion_rate} - w_3 \cdot \text{harm_score}$

Decoupled Objectives Formula:

Final Score = $w_1 \cdot \text{engagement} + w_2 \cdot \text{quality} - w_3 \cdot \text{harm}$

Weights are **tunable business knobs** — no retraining needed to change policy. Train models separately; combine at serving.

FAANG Probe: "You've built a news feed ranker optimized for CTR. T&S says it's amplifying misinformation. Fix it without rebuilding." → Decouple — add a **separate harm classifier** and quality scorer; re-rank with weighted multi-signal formula at serving. Don't retrain CTR model; add guard rails as new signals.

Chapter-Specific Requirement Triggers

NEW

SYSTEM	CRITICAL REQUIREMENT TO CLARIFY FIRST	RISK IF MISSED
Event Rec (Ch7)	Location radius (geofencing constraint), event time window urgency, social graph access	Model recommends expired events or events 500km away; wasted registrations
Ad Click (Ch8)	Revenue vs. CTR optimization conflict; advertiser fairness constraints (equal ad exposure)	CTR model maximizes clicks but not revenue; small advertisers get zero impressions
Similar Listings (Ch9)	Session context — vacation search is session-bounded, not based on long-term user history	Model uses stale city preferences; recommends listings in a city the user visited 6 months ago
PYMK (Ch11)	Privacy constraints on graph traversal depth; mutual friend access policies	Traversing 3rd-degree connections without consent → GDPR/privacy violation
Street View Blurring (Ch3)	Recall > Precision requirement — must never miss a face ; false negatives are legal violations	Optimizing for precision misses faces → GDPR liability; no acceptable FN rate
Harmful Content (Ch5)	Human review capacity as constraint on threshold bands; reviewer SLA	Too-low threshold floods human review queue; system overwhelms ops capacity

Business Objective → ML Objective Translation (All 10 Systems)

v32 · Fixed

Metric Chain (read left → right): North Star constrains which → Proxy Metric is valid; proxy type determines → Loss Function ; after training you measure with → Offline Eval Metric

Key Distinction: Proxy Metric = the ML-level metric you try to maximise/minimise as a stand-in for the North Star (e.g., CTR, watch completion rate). It determines what type of output you need → which fixes the loss function. **Offline Eval** = statistical tool computed after training to verify the model achieved the proxy (e.g., AUC, nDCG). **Guard Metrics** = what must NOT degrade even when proxy improves. Together, proxy + guard define the full constraint space before you pick a loss.

SYSTEM	BUSINESS OBJECTIVE	ML OBJECTIVE	▲ NORTH STAR	■ GUARD METRIC
Visual Search Pinterest	Help users discover images	Rank images by visual similarity to query	DAU engaged w/ visual search; % searches → save/pin	p99 latency result quality Recall@k brute force
Street View Blurring Google	Protect privacy	Detect faces & plates; maximise Recall	Zero GDPR violations; privacy complaint rate → 0	FNR per user tone (e.g., wall) latency
YouTube Video Search	Surface relevant videos	Rank videos by text-query relevance	7-day search-driven retention; total watch time from search	p99 latency result diversity
Harmful Content Meta	Improve platform safety	Maximise harm detection rate (Recall)	Harmful content prevalence rate; user trust score	Human capacity demographics
Video Rec YouTube	Increase engagement	Maximise watch time + relevancy score	7-day retention; total weekly watch time per user	Content quality latency across devices rate
Event Rec Eventbrite	Increase ticket sales	Maximise # of event registrations	Total ticket revenue; # registrations per session	No explicit events latency impressions

SYSTEM	BUSINESS OBJECTIVE	ML OBJECTIVE	▲ NORTH STAR	■ GUA
Ad Click Prediction	Increase revenue	Maximise CTR / predicted revenue	Total ad revenue; advertiser ROI	Hide min in advert
Similar Listings Airbnb	Increase bookings	Maximise session booking conversion	Session booking conversion rate	Session listing latency
News Feed Facebook	Increase engagement	Maximise weighted engagement score	7-day DAU; time spent / session; long-term retention	Content harmful rate;
PYMK LinkedIn	Grow user networks	Maximise connections formed	# accepted connections / week; network growth rate	Graph (GDPR) accep

◆ **Proxy — Interview move:**

"My proxy is [X] because it correlates with the North Star and is directly measurable from user behaviour. Its type — binary/continuous/ranking — determines my loss function."

■ **Guard — Interview move:**

"I'd also track [guard metric] because aggressively maximising [proxy] without it causes [specific failure mode]."

● **Offline Eval — Interview move:**

"I'll gate deployment with [offline metric] because this task is [ranking/classification/retrieval] and [metric] best captures [ranking quality / calibration / precision-recall tradeoff]."

FAANG Trap — Three separate things: Ad Click: **Proxy** = CTR (binary, $P(\text{click})$) → **Loss** = BCE → **Offline Eval** = ROC-AUC + NCE → **North Star** = Revenue. None of these is the same thing. Conflating proxy with training label or offline eval is a mid-level signal.

Combining Loss Functions & Serving Scores – Master Equations

Staff Critical • NEW

① Universal Multi-Task Loss (News Feed, Harmful Content, Video Rec)

Combine K independent task losses into one backward pass. Weights are **tunable without retraining** — this is the business policy knob.

$$\mathcal{L}_{\text{total}} = \sum_{k=1}^K w_k \mathcal{L}_k \quad \text{where} \quad \sum_{k=1}^K w_k = 1$$

SYSTEM	K	TASK HEADS	LOSS PER HEAD
Harmful Content	4–6	violence, nudity, hate, self-harm, spam	BCE per label (independent, not softmax)
News Feed	5	like, share, comment, hide, report	BCE per head; w_{hide} & w_{report} negative
Video Rec	2	click (binary), watch-time (float)	BCE + Huber; watch-time weighted 5–10×

Trap: Using Softmax across task heads implies mutual exclusivity — a post cannot be both a like and a share. Always use **independent BCE per head** for multi-label tasks.

② Final Serving / Re-ranking Score — Decoupled Objectives Pattern

Models are trained separately. Scores are combined at **serving time** — no retraining needed to change policy weights.

$$\text{Score}(u,i) = w_1 \cdot \hat{p}_{\text{click}} + w_2 \cdot \hat{t}_{\text{watch}} - w_3 \cdot \hat{p}_{\text{harm}} + w_4 \cdot \text{diversity_bonus}(i)$$

SYSTEM	SERVING SCORE FORMULA	KEY DESIGN DECISION
Video Rec	$w_1 \cdot P(\text{click}) + w_2 \cdot E[\text{watch_time}] - w_3 \cdot P(\text{harm})$	w_2 typically 5–10× w_1 (watch time is the north star proxy)
News Feed	$w_1 \cdot \text{like} + w_2 \cdot \text{share} + w_3 \cdot \text{comment} - w_4 \cdot \text{hide} - w_5 \cdot \text{report}$	hide/report get negative weights to suppress harmful content
Ad Click	$\text{eCPM} = \text{bid} \times \hat{P}(\text{click}) \times \hat{P}(\text{conversion})$	$\hat{P}(\text{click})$ must be calibrated — sets actual bid price in auction
Event Rec	$w_1 \cdot P(\text{click}) + w_2 \cdot P(\text{register}) - \text{penalty}(\text{distance} > \text{radius})$	hard penalty on geo/time constraints before soft score

③ Contrastive & Embedding Losses (Visual Search, YT Search, Similar Listings)

NAME	FORMULA	SYSTEMS	KEY PARAM
NT-Xent / SimCLR	$\mathcal{L} = -\log \frac{e^{\text{sim}(z_i, z_j)}}{\sum_{k \neq j} e^{\text{sim}(z_i, z_k)}}$	Visual Search	$\tau=0.07$ typical; larger batch = more negatives
Sampled Softmax	$\mathcal{L} = -\log \frac{e^{s_{u,i}}}{\sum_{j \in \mathcal{B}} e^{s_{u,j}}}$	YT Search, Video Rec two-tower	In-batch negatives; item freq correction
BPR	$\mathcal{L} = -\sum \log(\sigma(s_{ui} - s_{uj}))$	Similar Listings (Airbnb)	i = positive click; j = in-session negative

④ Imbalanced Classification Losses (Street View, Ad Click, Harmful Content)

$$\mathcal{L}_{\text{focal}} = -\alpha_t (1 - \hat{p}_t)^{\gamma} \log(\hat{p}_t)$$

VARIANT	ADDS	PICK WHEN	SYSTEM
Plain BCE	—	Balanced classes; calibration needed	Ad Click, PYMK, Event Rec
Focal ($\gamma > 0$)	$(1 - \hat{p})^{\gamma}$ down-weights easy negatives	Positive rate <1%; rare object detection	Street View, Ad Click (rare clicks)
Weighted BCE	$\alpha_k = N_{\text{neg}}/N_{\text{pos}}$ per class	Class freq imbalance; multi-task rare heads	Harmful Content (self-harm 0.01%)
Focal + Weighted	Both α and γ	Freq gap AND easy-negative dominance	Harmful Content multi-task DNN

⑤ Calibration Check – Normalised Cross-Entropy (NCE) – Ad Click Critical

Log-loss alone does not tell you if $\hat{P}(\text{click})$ is well-calibrated. NCE normalises by a prior-only baseline — if $\text{NCE} < 1$ your model is **worse than a prior** and will misprice bids.

$$\text{NCE} = \frac{-\frac{1}{n} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]}{-(p \log p + (1 - p) \log(1 - p))}$$

p = average empirical CTR (prior baseline). $\text{NCE} = 1 \rightarrow \text{model} \equiv \text{prior}$. $\text{NCE} < 1 \rightarrow \text{worse than prior}$. $\text{NCE} > 1 \rightarrow \text{adds predictive value beyond base rate}$.

Interview Ritual: When asked "how do you set the loss function?", say: "I first identify if the output is binary, multi-label, or continuous — that fixes the loss type. Then I check class balance (focal vs. weighted BCE). For multi-task systems I use a weighted sum of per-head losses with tunable weights. Finally, for ad systems I audit calibration with NCE because $\hat{P}(\text{click})$ directly prices the bid."

Interview Ritual: After reading the prompt, say: "Before I design anything, let me clarify a few things." Fire 3–4 questions from the 7 dimensions. Then **proactively raise RSMA** (reliability, scalability, maintainability, adaptability) — this alone distinguishes Staff-level candidates. Spend 3–4 minutes here. Skipping it is an immediate red flag.

Step 2 · Problem Framing

Translate business goal into a trainable ML task – all major category decisions live here

PRE-GATE – Business Framing

Define objectives before any architecture decision

When NOT to Use ML

Staff Opener · NEW

Staff+ Opener: Start your answer with: "Before jumping to ML, let me verify ML is the right tool here." This signals ownership and system-design maturity. Most candidates assume ML is required – don't be that candidate.

ASK YOURSELF	IF YES → ACTION
Is the pattern too simple for ML? (e.g., zip code → state)	Use a lookup table or rule-based system
Is labeled data available or feasible to collect at scale?	No labeled data → rule-based or heuristic baseline first
Does the cost of being wrong warrant ML complexity?	Low-stakes task → rules > ML overhead
Can a rule-based system achieve 80% of value with 10% engineering cost?	Ship rules first; evolve to ML when rules hit ceiling
Is the feature space clearly non-linear and high-dimensional?	ML justified – rules cannot generalize

Generic Framework for Unknown Use-Cases

Universal · NEW

When facing a novel prompt (e.g., "Design a fraud detection system" or "Doctor appointment recommender"), apply this 5-step mental checklist before proposing any architecture.

STEP	QUESTION	WHY IT MATTERS
1. Pattern Match	Is this a ranking, classification, retrieval, or generation problem? Almost every FAANG ML design reduces to one of these four archetypes.	Determines model family, loss function, and serving pattern
2. Two-Stage Rule	If item space >100K, propose retrieval + ranking . Always. Then justify with latency math.	Gates architecture. Cannot skip without QPS justification.
3. Feedback Signal	What is the natural label ? Click? Purchase? Booking? Time spent? Report? This drives training data strategy and determines position bias exposure.	Dictates labeling window, debiasing technique, and data freshness requirement
4. Cold Start	Every system has a cold start. Identify it: new user, new item, or both . Name your mitigation before the interviewer asks.	Unaddressed cold start = system unusable at launch for new entities
5. Failure Modes	Name all 4 failure modes proactively: degenerate feedback loop, distribution shift, adversarial inputs, fairness/exposure bias	Proactive failure naming signals ownership thinking and production readiness

☆ **Staff+ Badge:** A Staff candidate names failure modes before being asked. This demonstrates you're designing for production failure, not just the happy path.

Universal FAANG Probe: "Your model is live. After 3 months, engagement is flat despite improved offline metrics. What do you investigate?" → (1) Training-serving skew — are features consistent between training and inference? (2) Distribution shift — is today's data representative of training data? (3) ML objective no longer correlates with business north star. (4) Degenerate feedback loop — model only recommends popular items, reducing exploration.

GATE 1 – Scale Gate ————— Item space > 100K? → Two-Stage mandatory

Two-Stage Framing: Retrieval + Ranking

Used in 5 of 8 Systems • NEW

Default Pattern for Large Item Spaces: If the candidate item count >100K, always propose two-stage architecture before the interviewer asks. This is the canonical production design at YouTube, TikTok, LinkedIn, and Airbnb.

STAGE 1 – RETRIEVAL

- › **Goal:** Billions → Hundreds (reduce candidate space)
- › **Optimize for:** Recall — accept false positives
- › **Model:** Lightweight dual-encoder / ANN similarity search
- › **Feature budget:** ID embeddings only (no dense features)
- › **Latency:** <20ms via pre-computed item embeddings
- › **Tools:** Faiss, ScaNN, HNSW index

STAGE 2 – RANKING

- › **Goal:** Hundreds → Top-K (precision scoring)
- › **Optimize for:** Precision — quality over speed
- › **Model:** Heavy cross-attention / GBDT with full feature set
- › **Feature budget:** 100s of features (user, item, context, cross)
- › **Latency:** <100ms over small candidate set
- › **Tools:** DeepFM, Two-tower with feature crossing, XGBoost

☆ **Staff+ Badge:** Propose the two-stage architecture proactively, then justify: "We can't run a 200-feature DNN against 50M items per request. Retrieval gives sub-20ms latency; ranker provides quality over the shortlist."

FAANG Probe: "Why can't you use a single heavy model for both retrieval and ranking?" → Latency and compute. A cross-encoder computing attention against 50M candidates takes minutes. Two-stage trades recall for speed in Stage 1 (ANN allows approximate misses), then compensates with precision in Stage 2 over ~500 candidates.

GATE 2 – Signal Gate ————— What interaction signal / data type do you have?

Gate 2 – Signal → Paradigm + Output Framing

Unified · NEW

SECTION A – Stage 1 Retrieval: Choose by Signal Type Billions → hundreds of candidates

SIGNAL AVAILABLE	RETRIEVAL ARCHITECTURE	HOW IT SCORES CANDIDATES	COLD START	SYSTEMS
Rich interaction history, no new users/items	Matrix Factorization	Dot product of learned ID embeddings	X fails new users	Video Rec candidate gen
Cold start exists — new users & items	Two-Tower NN	$\text{cosine}(\text{user_features_emb}, \text{item_features_emb})$	✓ uses feature towers	Video Rec, PYMK
Session-bounded intent > long-term history	Session-based (W2V)	Co-occurrence similarity in session window	✓ 2–3 clicks sufficient	Similar Listings (Airbnb)
Social graph (friends-of-friends) available	GNN edge prediction	Edge probability from neighbor aggregation	⚠ partial — profile features help	PYMK (LinkedIn)
No interaction data — images or text content only	Representation Learning	Contrastive loss (SimCLR) + ANN nearest neighbor	N/A — no user concept	Visual Search, YT Search
Fixed, small output space (<10K items, never grows)	N/A — Multiclass only	Softmax over N fixed classes — no retrieval stage needed	N/A	News category prediction

⚡ BYPASS CONDITION — Sparse Tabular Features (no sequence, no graph)

When your interaction data is a flat user × item × context feature table — high-cardinality categoricals like user_id, ad_id, device, hour_of_day, bid_price — the two-stage retrieval pattern does **not apply**.

- › **Skip Stage 1 entirely** — ad targeting rules act as the candidate filter, no ANN needed
- › **Go directly to DeepFM** as a single-stage ranker — FM layer handles sparse pairwise interactions cheaply; DNN captures high-order interactions
- › **Why not Two-Tower?** Two-Tower requires separable user/item towers for ANN pre-computation. Tabular features are dense cross-features (user × ad × context) — they can't be split into independent towers
- › **Why not GBDT here?** Valid for batch/offline, but eliminated if retraining is continuous (Ad Click 5-min staleness window)

System example: Ad Click (Ch8) — user_id + ad_id + context → DeepFM → $P(\text{click}) \in [0,1]$ → bid price calculation

GATE 3+4 – Stage 2 Ranking ————— Label type → training paradigm + loss · Output count → heads

Stage 2 Ranking – Unified Decision Card

☆ STAFF+

LTR = training paradigm, not a model, not a stage. Label type forces LTR paradigm → forces loss function. Stage 2 model is always Heavy DNN or DeepFM. Gates 3 and 4 are **independent axes**: Gate 3 = what math to minimise per head · Gate 4 = how many heads.

3A – Label → LTR Paradigm → Loss Hundreds → Top-K · all features concatenated at layer 1

LABEL TYPE	LTR PARADIGM	LOSS	OUTPUT	SYSTEMS
Binary click / no-click (most common)	Pointwise — score each item independently	BCE	$P(\text{click}) \in [0,1]$	Ad Click, Event Rec, News Feed

LABEL TYPE	LTR PARADIGM	LOSS	OUTPUT	SYSTEMS
Continuous float — watch time, dwell, price	Pointwise — regress each item independently	Huber / MSE	Predicted value → sort	Video Rec, Similar Listings
Graded relevance 0–3, editorial judgments	Pairwise — learn "A > B" from label pairs RankNet · LambdaRank	LambdaRank (optimises nDCG)	Relevance score → rank	YT Video Search
Complete item ordering, full list judgments	Listwise — optimise full list quality SoftRank · ListNet · rare in prod	ListNet CE	Full ranked list	Advanced search only

Paradigm selector (ask in order):

- ① Binary clicks only? → **Pointwise + IPS**
 - ② Graded labels 0–3? → **Pairwise**
 - ③ Full ordering? → **Listwise**
- Never pick Pairwise just because "ranking quality matters" — you need the label type first.

Two-Tower vs Heavy DNN:

Two-Tower → Stage 1 only. Towers kept separate for ANN pre-computation. No cross-feature learning.
Heavy DNN → Stage 2 only. All features concatenated at layer 1. Learns user × item × context interactions.

Position bias: Clicks at rank 1 ≠ relevance. Fix with **IPS** — reweight each sample by $1/P(\text{shown at position } k)$ before training.

FAANG Probe: 500M video catalog, click labels, cold start matters.

Stage 1: Two-Tower + Faiss ANN → ~500 candidates · Stage 2: Heavy DNN, BCE, Pointwise LTR → P(click).

Not Pairwise (no editorial labels) · Not Multiclass (500M outputs infeasible) · Watch time label → swap BCE → Huber

3B — Output Count → Single-Task vs Multi-Task Each head can have a different loss (BCE + Huber in same model)

CONDITION	✓ MULTI-TASK (MTL)	SINGLE-TASK
Label scarcity	Data-sparse tasks borrow signal via shared backbone self-harm 50K ← borrows from violence 5M	Each task has abundant labels — no sharing benefit
Semantic overlap	Tasks share representation violence / nudity / hate → all need "harm"	Tasks dissimilar → shared layers hurt individual tasks
Inference cost	One forward pass → N outputs News Feed: like + share + comment + dwell	N models = N inference calls + N pipelines
Combined loss	$L = w_1 \cdot L_1 + w_2 \cdot L_2 + \dots$ · Each head: own loss type · Weight tuning = main engineering challenge	

FAANG Probe: Content moderation — 5M violence labels, 50K self-harm labels. One model or separate?

→ **MTL:** shared backbone transfers harm signal. Self-harm head gets focal loss for 100:1 imbalance. One inference call covers all harm categories.

Classification Architecture Choices

OPTION	WHEN	TRADE-OFF	EXAMPLE
Single binary classifier	One harm type; fast to build	Can't explain which harm class	Baseline only
	Each class needs independent tuning		

Step 3 · Data Preparation

Labeling · sampling · imbalance · encoding · modality transforms · data lineage

Labeling Strategy Decision

STRATEGY	WHEN TO USE	TRADE-OFF
Hand labeling	High-quality labels needed; ambiguous content; eval sets	Expensive, slow, annotator bias
Natural labeling (clicks, bookings, likes)	Large scale; fast iteration	Noisy; sparse for rare events
Self-supervision / augmentation	No interaction data; image tasks (SimCLR / MoCo style)	No upfront cost; data differs from real-world similarity
Hybrid ✓	Best of both worlds	Start with natural labels → human review to remove noise

Book examples: Visual Search uses self-supervision (data augmentation); Harmful Content uses hand labeling for eval, natural (user reports) for training.

Feedback Loop Length & Labeling Windows Staff+

Feedback loop length is a **first-class data design decision** — define a labeling window per system before training begins.

LOOP TYPE	WINDOW	RISK	EXAMPLE
Short loop	Minutes	Noisy; user may regret click immediately	CTR on search results
Medium loop	Hours–days	Moderate noise; acceptable for most tasks	News Feed reactions
Long loop	Days–months	Premature negatives if window too short	Bookings (Airbnb), fraud

Premature negative: User clicked but did not book within 1-day window → mislabeled as negative. Fix: use a 3–7 day window or probabilistic labels for delayed conversions.

Label Multiplicity & IAA (Inter-Annotator Agreement) Staff+

For ambiguous tasks (harm detection, content quality), the same item may receive different labels from different annotators — a real production problem at FAANG scale.

METHOD	DETAIL
Cohen's kappa (κ)	IAA metric; $\kappa > 0.8$ = strong agreement; $\kappa < 0.6$ = unreliable labels — revisit guidelines
Majority voting	Use label agreed by $\geq 50\%$ of annotators; fast but loses minority signal
Expert arbitration	Route disagreements to senior reviewer; expensive but necessary for safety tasks
Soft labels	Use label distribution (e.g., 0.7 harmful / 0.3 not) instead of hard 0/1; produces more calibrated models

Weak Supervision & Programmatic Labeling Staff+

Active Learning Staff+

At scale you cannot hand-label everything. Use **LFs (Labeling Functions)** — heuristics, regex rules, distant supervision — to programmatically assign noisy labels (Snorkel-style).

CONCEPT	DETAIL
LF (Labeling Function)	Rule or heuristic that votes on a label; individual LFs are noisy but ensemble is robust
Denoising LFs	Train a generative model to reconcile conflicting LF votes → single probabilistic label
Privacy advantage	No annotator sees raw data — critical for PII-sensitive systems (medical, legal)
vs. active learning	Prefer weak supervision when labels can be expressed as rules; prefer active learning when model uncertainty guides what to label

FAANG probe: "You have 10M items you cannot manually label. How do you get training labels at scale?" → Weak supervision via LFs + denoising generative model (Snorkel-style)

Given 1M unlabeled items and a budget for 10K labels — **do not random sample**. Select the highest-information examples first.

STRATEGY	HOW	PICK WHEN
Uncertainty sampling	Label items where model is least confident — $P(y x) \approx 0.5$	Single model; fast iteration loop
Query-by-committee	Ensemble of models; label items where they disagree most	Model diversity available; more robust selection
Margin sampling	Select items where gap between top-2 class probs is smallest	Multi-class problems

Cycle: Train → predict on unlabeled pool → select uncertain items → label → retrain

Root cause first. All three techniques fix class imbalance, but attack different root causes. Ask these 3 questions in order before choosing a loss function.

Q1 – Do I know the business cost of FN vs FP?

YES → Cost-Sensitive Learning

Domain expert defines cost matrix C_{FN} vs C_{FP} . Use when FN costs 10×+ more than FP.
e.g., Insurance fraud, Medical diagnosis, Street View (FN = legal liability)

NO → continue ↓

Q2 – Are there millions of obviously-correct negatives drowning the gradient?

"Easy negatives" = examples the model already predicts correctly with high confidence (e.g., 99.9% non-clicks, cat photos in harm detection, clear non-fraud transactions)

YES → Focal Loss ($\gamma=2$ default)

$L = -(1-p)^\gamma \log(p)$. Mathematically silences easy examples. Model capacity goes entirely to hard, ambiguous cases.
e.g., Ad Click (0.1% CTR), Harmful Content (obvious safe posts dominate), Object Detection

NO → continue ↓

Q3 – Is it purely a frequency problem – too few labeled positives?

YES → Class-Balanced Loss

Weight = $n_{\text{majority}} / n_{\text{minority}}$. Makes model treat 50K self-harm examples as if they were 5M.
e.g., Multi-task harmful content (self-harm 50K vs violence 5M), any rare-event classifier as baseline

Production Combination Pattern – Stack in order of specificity:

Focal Loss (silences easy negatives per example) + Class-Balanced (fixes frequency gap across tasks in multi-task models) + Cost-Sensitive threshold at serving time (business policy, no retrain needed).

⚠ Never resample large transformer-scale models – resampling breaks calibration; $P(y|x)$ no longer matches the real-world class prior. Use two-phase training instead.

TECHNIQUE	ROOT CAUSE FIXED	PICK WHEN	CAUTION	SYSTEM EXAMPLE
Class-Balanced Loss	Frequency imbalance – too few positives in the dataset	Baseline fix for any imbalanced problem; multi-task label frequency gap	Still trains on easy negatives – just amplifies positives	Harmful Content (self-harm vs violence)
Focal Loss	Difficulty imbalance – easy negatives steal gradient	Positive rate <1%; object detection; sea of obvious correct negatives	γ hyperparameter needs tuning per task	Ad Click (0.1% CTR), Street View Blurring
Cost-Sensitive	Business-cost asymmetry – FN and FP have different consequences	FN cost >> FP cost; domain expert can quantify the ratio	Cost matrix must come from domain experts; hard to estimate automatically	Insurance fraud, Medical diagnosis
				Harmful Content multi-

TECHNIQUE	ROOT CAUSE FIXED	PICK WHEN	CAUTION	SYSTEM EXAMPLE
Focal + Class-Balanced	Both frequency gap AND easy negative dominance	Multi-task model with rare tasks AND large-scale imbalanced data	Two hyperparams to tune: γ and per-class weights	task DNN (Meta)
Two-Phase Learning	Calibration break from resampling	Large model (transformer) where $P(y x)$ calibration is critical	Phase 2 LR must be low or Phase 1 gains overwritten	Ad auction CTR – bid price = $P(\text{click})$

FAANG Probe: "You have 99.9% non-clicks in your ad click training data. Which loss function do you choose and why?"

→ Focal Loss ($\gamma=2$). Root cause is easy-negative dominance — 999M non-click impressions are trivially correct ($p \approx 0.001 \rightarrow \text{loss} \approx 0.001$ each). Class-balanced would amplify positives but still waste 80% of gradient on easy negatives. Cost-sensitive requires a business cost matrix we don't have. Focal silences easy negatives mathematically. Also add two-phase training to preserve calibration for bid pricing.

Sampling Strategies

Staff+ · Completely absent at Senior level

STRATEGY	HOW	WHEN TO USE	TRADE-OFF
Random sampling	Uniform random draw from population	Balanced, large datasets; baseline approach	Underrepresents rare classes
Stratified sampling	Sample each class/group proportionally or equally to ensure representation	Rare-class tasks (object detection, fraud, harmful content); class-imbalanced data	Requires knowing class membership ahead of time
Reservoir sampling	Maintain fixed-size window over stream; each new item has k/n probability of inclusion	Streaming production data; real-time retrain pipelines; unknown dataset size	Memory bounded; $O(1)$ per item — efficient for high-throughput pipelines
Importance / weighted sampling	Assign higher sampling prob to underrepresented distributions; reweight by $P(\text{train})/P(\text{serve})$	Distribution mismatch between training and serving; correcting selection bias in logged data	Requires estimating serving distribution; can increase variance
Temporal split ✓	Train on past data; validate/test on future data — never random-shuffle across time	All production ML systems — prevents data leakage from future signal	Less training data; distribution shift if seasonality exists

FAANG probe: "How do you ensure your training data is representative of production?" → Stratified sampling for rare classes + temporal splits to prevent leakage + importance weighting to correct selection bias from logged data

Class Imbalance Handling – Staff-Level Depth

Staff+

TECHNIQUE	HOW	PICK WHEN	CAUTION
Oversampling	Duplicate minority class samples	Small dataset; minority very rare	Risk of overfitting to duplicated samples
Undersampling	Remove majority class samples	Large dataset; accept data loss	Discards potentially useful majority data

TECHNIQUE	HOW	PICK WHEN	CAUTION
SMOTE (Synthetic Minority Oversampling Technique)	Generate synthetic minority samples by interpolating between real minority examples in feature space	Tabular data; small minority class; need more diversity than simple duplication	Can generate unrealistic samples; poor on sparse high-dimensional features
Focal loss	Down-weight easy negatives: $L = -(1-p)^\gamma \cdot \log(p)$; higher γ = more focus on hard examples	Object detection; harmful content detection; imbalanced classification	Hyperparameter γ needs tuning per task
Class-balanced loss	Weight each class loss by inverse of its frequency	General imbalanced classification; good baseline fix	Sensitive to class frequency estimation
Two-phase learning	Phase 1: train on resampled/balanced data → learn decision boundaries. Phase 2: fine-tune on original unmodified distribution → fix calibration	Large models with calibration requirements (fraud, ads); when simple resampling hurts probability estimates	Phase 2 LR must be low or Phase 1 gains are overwritten
Cost-sensitive learning	Define cost matrix C_{ij} : cost of predicting class j when true class is i ; asymmetrically penalizes misclassifications	Fraud detection (FN far worse than FP); safety systems; asymmetric error costs	Cost matrix must be specified by domain experts; hard to estimate automatically
⚠ Do NOT resample if...	Model is large enough (transformer-scale) and trained on original distribution	—	Resampling hurts calibration — output $P(y x)$ will no longer match real-world class prior; use cost-sensitive learning instead

FAANG probe: "Your fraud model has 99.9% accuracy but flags nothing. How do you fix it?" → Cost-sensitive learning with high C_{FN} + two-phase training to preserve calibration. Resampling alone will hurt probability estimates.

Missing Value Types — Know Why Data Is Missing

Chip Huyen NEW

Not all missing values are equal. The **type** of missingness changes how you should handle it — and confusing them causes silent production bugs. Name the type before proposing a fix.

TYPE	DEFINITION	EXAMPLE	CORRECT ACTION
MNAR Missing Not At Random	Missing because of the value itself — the absence is correlated with the true value	High earners skip income field; users with poor engagement hide their activity	Never just impute. The absence IS a signal. Add a binary indicator feature: <code>is_income_missing = 1</code>
MAR Missing At Random	Missing due to another observed variable — not the missing value itself	Gender A users never fill the age field; device type X never logs location	Safe to impute — but impute within the group (conditional imputation), not globally
MCAR Missing Completely At Random	Truly random — no relationship to observed or unobserved data. Rare in practice.	Random sensor dropout; intermittent logging failure	Row deletion or mean/median imputation acceptable — but verify randomness first

Interview Rule: When an interviewer asks "how do you handle missing values?", **never just say mean imputation.** Say: "I first identify the missing type. For MNAR, the absence is a signal — I'd add an `is_field_missing` indicator. For MAR, I'd use conditional group imputation. Only for MCAR is row deletion acceptable."

FAANG Probe: "Your model's income feature has 30% missing values. What do you do?" → Ask why it's missing. If high earners skip it (MNAR), adding a binary `is_income_missing` feature is more valuable than any imputation. Imputing MNAR data introduces systematic bias and destroys the signal.

Missing Value Handling

METHOD	PICK WHEN	RISK
Column deletion	>50% missing	Loses signal
Row deletion	Many features missing in one row	Reduces training size
Imputation (mean / median / mode)	Missing at random; small fraction	Introduces noise

Feature Scaling

METHOD	PICK WHEN
Min-max normalization [0,1]	No outliers; bounded range needed
Z-score standardization $N(0,1)$	Outliers manageable
Log scaling	Skewed distributions (counts, durations)

Categorical Encoding

METHOD	PICK WHEN	LIMIT
Integer encoding	Ordinal relationship exists	Implies false ordering
One-hot encoding	Low cardinality (<100)	Sparse if high cardinality
Embedding learning	High cardinality; semantic meaning needed	Requires training data

Data Leakage – 4 Silent Sources That Kill Production Models

Chip Huyen NEW

Data leakage = future information enters training data, inflating offline metrics but causing silent failure in production. The model learns shortcuts that don't exist at serving time.

LEAKAGE SOURCE	HOW IT HAPPENS	FIX
1. Scaling before splitting	Computing mean/variance on the full dataset (train + test) leaks test statistics into training	Always split first , then fit the scaler exclusively on the train split. Apply to test.
2. Imputing before splitting	Filling missing values using median/mean computed on full data — test distribution leaks into train	Split first , then compute imputation statistics on train only. Transform test with train stats.
3. Temporal leakage	Random shuffling time-series data — model trains on events after the prediction target	Always use temporal splits : train on past, validate/test on future. Never random-shuffle across time.
4. Process leakage	A feature correlates with the label not through causality but because it was generated after the label event (e.g., scan machine tag assigned post-diagnosis)	Audit feature creation timestamps. Ensure every feature in training was available before the label event.

Leakage Detection Checklist:

- › Suspiciously high offline metrics (near-perfect AUC)? → First check for leakage before celebrating
- › Feature importance dominated by one unexpected feature? → Check its creation timestamp vs. label event
- › Offline metrics collapse when you switch to a strict temporal split? → You had temporal leakage
- › New items/users score perfectly at launch but degrade fast? → Process leakage or future-dependent feature

FAANG Probe: "Your fraud model has 99.5% AUC offline but barely beats random in A/B test. What happened?"
→ Classic leakage. Check: (1) Was data scaled/imputed before splitting? (2) Was there random shuffling of time-ordered fraud events? (3) Does any feature encode post-fraud-event metadata (e.g., account status updated after fraud was confirmed)?

Data Lineage – Staff System Design Signal

Staff+

Data lineage = tracking the origin of every data sample and its label through the entire ML pipeline. Critical for debugging model regressions after data updates.

COMPONENT	WHAT TO TRACK	WHY IT MATTERS
Data source tag	Which upstream system produced each sample (crawler, user event, annotation tool)	Isolate which source caused a regression after adding 1M new samples
Label provenance	Annotator ID, LF name, rule version that generated the label	Trace labeling errors; audit annotation quality; remove systematically mislabeled batches
Timestamp	When sample was created and when label was assigned	Enables temporal slicing for debugging; ensures train/test split integrity
Version control	Dataset version and feature schema version that produced each model	Reproduce any model from any checkpoint; compare model versions apples-to-apples

FAANG probe: "Your model degraded after adding 1M new samples. How do you debug it?" → Data lineage: tag all samples by source → slice eval metrics by source → identify which batch introduced the noise

Feature Crossing – When to Do It and Why

Chip Huyen NEW

Feature crossing creates new features from combinations of existing ones (e.g., `marital_status` × `num_children`). Whether you need it depends entirely on your model family — candidates who apply it blindly signal shallow knowledge.

MODEL FAMILY	NEED FEATURE CROSSING?	WHY
Logistic Regression / Linear models	✓ Essential	Linear models cannot learn non-linear interactions. Without crossing, <code>marital_status=married AND num_children=3</code> is invisible to the model. You must construct it manually.
GBDT / XGBoost	Sometimes helpful	Trees naturally split on feature combinations — but explicit crossing on high-signal pairs can speed up convergence and reduce tree depth needed.
Neural Networks (DNN, DeepFM)	Less critical — but explicit crossing still helps	DNNs learn interactions implicitly. But DeepFM/DCN explicit crossing accelerates learning of low-order interactions and improves sample efficiency.

Caveat to always raise: Feature crossing **blows up the feature space** — $100\text{-cardinality} \times 100\text{-cardinality} = 10,000$ combinations. This risks overfitting, memory blowup, and training instability. Use embedding-based crossing (FM, DeepFM) to handle high-cardinality crosses efficiently.

FAANG Probe: "Would you use feature crossing for your news feed ranker?" → For the DNN backbone: crossing is handled implicitly and by the FM component of DeepFM. For ablation: test whether explicit crosses on `user_age` × `content_category` improve nDCG. Raise the cardinality risk proactively.

Feature Store – 3 Problems It Solves (Staff Badge)

Chip Huyen NEW

A Feature Store is not just an optimization — it is the **canonical fix for training-serving skew**, the most common Staff-level silent failure mode. Mention it proactively in any system with real-time serving.

#	PROBLEM SOLVED	DETAIL
1	Feature Management	Uber had 10,000 features across teams by 2017. Teams independently recompute the same features, wasting compute and creating inconsistencies. A Feature Store acts as a searchable, versioned feature catalog — teams discover and reuse existing features.
2	Feature Computation Efficiency	Expensive features (e.g., 7-day rolling watch time, social graph embeddings) should be pre-computed once and cached , not recomputed per request. Feature Store separates batch offline computation from online low-latency lookup.
3	Training-Serving Consistency (most critical)	Training pipeline uses Python/batch; serving uses Java/C++ streaming. Same feature computed differently → model sees different distributions at train vs. serve time. Feature Store enforces a single feature definition and transformation logic used by both pipelines.

Staff Badge: Proactively say: "To prevent training-serving skew, I'll use a Feature Store that enforces a single feature definition shared between the offline training job and the online inference service." Name tools: Feast (open source), Tecton, Vertex AI Feature Store, Databricks Feature Store.

FAANG Probe: "Your model works great in offline eval but degrades immediately in production. What's the first thing you check?" → Training-serving skew: feature pipeline divergence. Are features computed identically in training and serving? Check Feature Store logs — are logged features from serving time identical to what the model saw during training?

Categorical Encoding – The Dynamic Category Trap

Chip Huyen NEW

Encoding brand/category/item as a fixed integer ID silently breaks in production when **new categories appear continuously**. This is the most common categorical feature production bug.

APPROACH	PROBLEM IN PRODUCTION	FIX
Integer ID encoding (brand=3, brand=47...)	New brand at inference → out-of-vocabulary → undefined behavior or crash. Model learned nothing about unseen categories.	Never use raw IDs for open-vocabulary categoricals.
One-hot (top-N + UNKNOWN bucket)	Acceptable if N is stable. But model must be trained on UNKNOWN — if it never sees UNKNOWN during training, it learns nothing for OOV categories.	Ensure training data includes UNKNOWN examples. Reserve 5–10% of training for UNKNOWN bucket.
Embedding layer ✓	New categories get their own embedding initialized randomly — but with a warm-start from similar seen categories via fallback.	Best for high-cardinality open-vocabulary categoricals. Generalizes via embedding similarity. Required for cold-start new items.
Hashing trick (feature hashing)	Maps any string to a fixed-size bucket — handles new categories automatically with no vocab update needed.	Hash collisions possible. Use for very high cardinality (hashtags, user IDs) where collisions are tolerable.

Production Rule: For any categorical feature where new values appear post-deployment (brands, advertisers, event hosts, content creators), **always use embeddings or hashing**. One-hot with fixed vocab requires a model redeploy every time a new category appears.

Ablation Studies – Feature Importance at Staff Level

Chip Huyen NEW

An **ablation study** systematically removes components (features, modules, loss terms) one at a time and measures the performance drop. It is the rigorous way to validate that your feature engineering choices actually help — and to prune dead weight before serving.

FINDING	WHAT IT TELLS YOU	ACTION
Removing feature X causes significant metric drop	Feature X is load-bearing. Model depends on it.	Invest in feature quality, freshness, and monitoring for X. Add drift alert for X's distribution.
Removing feature X causes no change in metric	Feature X is dead weight — redundant, noisy, or already captured by another feature.	Drop it. Reduces serving latency, memory, and potential leakage risk.
Removing feature X improves metric	Feature X is causing harm — possibly leakage, spurious correlation, or adversarial noise.	Investigate immediately. Check creation timestamp vs. label event. Check for leakage source.

Staff Badge: After proposing your feature set, say: "Post-training, I'd run ablation studies to validate which features actually contribute. Features that don't improve performance add serving cost and can introduce leakage risk — I'd prune them aggressively." This signals production-first thinking, not just accuracy-chasing.

FAANG Probe: "You added 50 new features to your ranker. Offline nDCG improved 1.2%. How do you decide which features to keep?" → Ablation: remove each feature group, measure nDCG delta. Keep features where

removal causes >0.1% drop. Drop features with zero or negative contribution. Separately flag any feature where removal improves the metric — investigate for leakage.

Text Vectorization

METHOD	SPEED	SEMANTICS	PICK WHEN
BoW / TF-IDF	Fast	None	Baseline; hashtags (News Feed)
Word2Vec / Embedding lookup	Moderate	Partial	Word-level similarity needed
Transformer (BERT, DistilmBERT)	Slow	Context-aware ✓	Semantics critical (harm detect, news)

FAANG probe: "BERT is slow for online inference — how do you handle latency?" → Use DistilmBERT (distilled, multilingual)

Video Encoding Strategy

METHOD	SPEED	TEMPORAL UNDERSTANDING
Video-level (3D CNN / Transformer)	Slow	✓ Captures motion & actions
Frame-level (ViT + aggregation) ✓	Fast	X Misses temporal flow

Book choice: Frame-level (ViT) for YouTube Video Search — faster serving; temporal context not critical for search relevance.

Step 4b · Data Tables & Feature Engineering

Universal 3-table schema · 4-bucket feature taxonomy · encoding patterns · Staff-level signals

The 3 Universal Data Tables

Every ML system — regardless of domain — maps to exactly three tables. **Name them first** before any feature discussion.

TABLE	UNIVERSAL COLUMNS	WHAT VARIES BY DOMAIN
User Table	user_id, age, gender, city, country, language, timezone	Identical across all 10 systems
Item Table	item_id, metadata fields, timestamp_created	Domain-specific columns — see quick-ref below
Interaction Table	user_id, item_id, interaction_type, interaction_value, location, timestamp	interaction_type = your training label. timestamp governs labeling window & temporal split

Interview move: "I have a User table, an [Item] table, and an Interaction table. The interaction type — click / register / like / book — becomes my training label."

Domain-Specific Item Table Quick Reference

SYSTEM	ITEM TABLE KEY COLUMNS
Video Rec / YT Search	video_id, duration, language, tags, title, likes, views
Event Rec	event_id, host_user_id, category, description, price, location, datetime
News Feed	post_id, author_id, text, hashtags, images/video, timestamp
Ad Click	ad_id, advertiser_id, category, bid, target_demographics
Similar Listings	listing_id, city, price, amenities, host_id
PYMK	user_id as item; edge features: mutual_connections, profile_visits
Visual Search	image_id, embedding (no manual tags — self-supervised)

The 4-Bucket Feature Taxonomy – Universal Mental Model

Apply this 4-bucket framework to any unknown use-case and you will cover 90% of what the interviewer expects.

BUCKET	WHAT IT CAPTURES	ENCODING PATTERN
 User Features	Who is the requester? Demographics, history, context	Embedding for IDs; One-hot for gender; Bucketize for age
 Item Features	What is being ranked/classified? Content, metadata, popularity	Embedding for item ID; BERT/ViT for text/images
 Context Features	When/where/how is this happening? Device, time, location	One-hot for device; Bucketize for time-of-day, day-of-week

BUCKET	WHAT IT CAPTURES	ENCODING PATTERN
🔗 Cross / Affinity Features	How do user ↔ item relate historically? Interaction rates, similarity scores	Cosine similarity; engagement rate; delta from user average

Per-Bucket Feature Checklist – Use Live During the Interview

BUCKET	FEATURES TO NAME	ENCODING
👤 User	Age, gender, country/city, language; device, time-of-day, day-of-week, location; mean-pool of previously liked/clicked item embeddings; BERT/W2V mean-pool of search queries	Bucketize → one-hot (age); one-hot (gender/device); embedding (country, language); mean-pool (history)
📦 Item	item_id; category, subcategory, language; title/description/tags; images; videos; view/like/registration count; item age	Embedding (ID); embedding/one-hot (categorical); BERT or TF-IDF (text); ResNet/ViT (images); log-scale normalize (counts); bucketize (freshness)
🌐 Context	Time of day, day of week, device type; user current location (geo-based systems)	One-hot (time, device); lat/long or distance-to-item (location)
🔗 Cross / Affinity Most missed!	user_like_rate_for_author, user_register_rate_for_category; cosine(user_history_emb, item_emb); distance_to_event – user_avg_event_distance (delta); # friends who interacted with item	Ratio (interaction rates); cosine similarity; delta / deviation; count (social signals)

Interview Execution Checklist – Say This Out Loud

#	WHAT TO SAY	WHY IT SIGNALS SENIORITY
1	"I have a User table , a [Domain Item] table , and an Interaction table ."	Grounds problem in real data structures before any modeling
2	"For features I'll group them into User, Item, Context, and Cross buckets."	Structured taxonomy prevents blanking; shows completeness
3	"Embeddings for high-cardinality IDs, one-hot for low-cardinality, log-scale for counts, BERT for free text."	Encoding awareness = production awareness
4	"Static features go to the Feature Store (offline). Dynamic features — real-time registration count, remaining event time — computed online at query time ."	Batch vs. streaming distinction = Staff-level signal
5	"I'll also compute a delta feature: distance to this event vs. the user's average distance to previously attended events."	Cross/affinity features separate Staff from Senior

☆ The Staff-Level Signal – Say This Verbatim

"Static features like user demographics and item metadata are pre-computed and stored in a Feature Store. Dynamic features like real-time registration count or remaining event time are computed inline at ranking time to avoid staleness."

Why this matters: Candidates who conflate batch and real-time features signal they've never shipped a production recommendation system. This single distinction separates Staff from Senior in Google/Meta interviews.

Step 4 · Model Development

Part A: Decision Layer · Part B: Core Deep Dives (MF→Two-Tower→W2V→GNN→DeepFM→MTL) · Part C: Specialized

PART A — DECISION LAYER ————— Which model + which loss — decide this before any architecture deep dive

Model Architecture Decision Logic – Interview Flowchart

SIGNAL AVAILABLE?	MODEL CHOICE	REASON
Rich interaction history, no cold start	Matrix Factorization	Fast; efficient; no features needed
New users / cold start problem	Two-Tower NN	Uses user + item features for new entities
Short session > long-term preference	Session-based (W2V style)	Co-occurrence in session = similarity signal
Social graph data available (FoF)	GNN (GraphSAGE / GAT)	Graph structure encodes relationship topology
Sparse high-cardinality features + CTR	DeepFM	Handles sparsity; FM low-order + DNN high-order
Multiple correlated tasks; some data-sparse	Multi-task DNN	Shared layers allow knowledge transfer
Multimodal input where cross-modal = signal	Early Fusion Multi-task DNN	Single model detects harm in meme (image+text)
Object detection (bounding boxes)	Faster R-CNN → YOLO → DETR	2-stage = accurate; 1-stage = fast
Visual similarity / embedding retrieval	ResNet/ViT + Contrastive + ANN	SimCLR/MoCo train; Faiss/ScaNN retrieve
Need daily / continuous retraining	NN – not GBDT	GBDT cannot be incrementally fine-tuned
Structured data + baseline fast	LR → GBDT → XGBoost	Fast iteration; interpretable

Model Selection by Data Type

DATA TYPE	MODEL PROGRESSION	NOTES
Structured / tabular	LR → GBDT/XGBoost → DCN → DeepFM	Add complexity only if needed
Images	CNN (ResNet) → ViT	ViT better with large data
Text	BoW → Word2Vec → BERT/Transformer	Task complexity drives choice
Video	Frame-level ViT → 3D CNN → Video Transformers	Frame-level preferred for serving speed
Graph	GNN (GCN, GraphSAGE, GAT, GIN)	PYMK, social network tasks

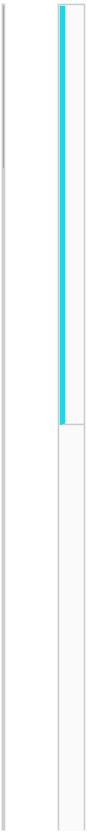


	C T	E n s	M fi	N h t

C T
I b re
R
U f s h E
U f o r ti le
E c s p
F s re s

C T	
C d d re	
R	
F r s	
F r p B	
L r o s	
R	
C e	

C T	
v tv	
N s V A	
S	
C d fe b	
N n h c	



Matrix Factorization — Deep Dive Ch. 6 — Video Recommendation

Decomposes user-item feedback matrix into **U** (user embeddings) × **V** (item embeddings). Relevance score = $\text{dot}(U_i, V_j)$

LOSS VARIANT	PROBLEM
Observed pairs only	All-ones embeddings → zero loss; overfits
All pairs	Sparse matrix → zeros dominate; predictions collapse to 0
Weighted combination ✓	$\sum(\text{obs})(A - UV^T)^2 + W \cdot \sum(\text{unobs})(A - UV^T)^2$ — W balances observed vs unobserved

OPTIMIZER	HOW	BEST FOR
SGD	Gradient descent over both matrices simultaneously	General purpose
WALS ✓ (Weighted Alternating Least Squares)	Fix U → optimize V; fix V → optimize U; alternate	Parallelizable → faster convergence

Pros: Fast training (only 2 embedding matrices); static embeddings → very fast serving (no transformation at query time)

Cons: Cannot use user/item features; cold start fails entirely for new users/items

Two-Tower Neural Network — Deep Dive Ch. 6 Video Rec • Ch. 10 PYMK

Architecture: [User features] → User Encoder (DNN) → User Embedding(d) ↔ [Item features] → Item Encoder (DNN) → Item Embedding(d) → dot product → similarity score

DECISION POINT	CHOICE	RATIONALE
Training objective	Binary cross-entropy	Positive = engaged; Negative = random/disliked
Negative sampling	Random + hard negatives	Hard negatives improve embedding quality
Candidate gen	Pre-compute item embeddings offline → ANN	User embedding at query time; item embeddings static
Ranking stage	Heavier model (more features)	Applied on ~1K candidates; accuracy > speed

DIMENSION	MATRIX FACTORIZATION	TWO-TOWER NN ✓
Training cost	Efficient	More expensive
Serving speed	Fastest (precomputed)	Slower (user embed at query time)
Cold start	X Fails	✓ Uses user/item features
Feature richness	X IDs only	✓ Demographics, context, content

Session-Based Embedding Model – Deep Dive Ch. 9 – Similar Listings / Airbnb

Maps each listing to an embedding so frequently co-occurring items within a browsing session are close in embedding space. Inspired by **Word2Vec**.

TRAINING VIA NEGATIVE SAMPLING + SLIDING WINDOW

Positive pair: (central listing, context listing in window) → label = 1

Negative pair: (central listing, random listing outside window) → label = 0

$$\text{Loss} = \sum(\text{pos}) \log \sigma(E_c \cdot E_p) + \sum(\text{neg}) \log \sigma(-E_c \cdot E_n)$$

CRITICAL ENHANCEMENT	HOW	WHY IT MATTERS
Global context – booked listing	Eventually-booked listing stays in the context window throughout training	Embeddings learn booking intent, not just click co-occurrence
Hard negatives – same-region listings	Add same-region non-clicked listings as hard negatives	Forces model to learn finer-grained distinctions within a city/neighborhood

Best for: Anonymous users; short-term intent > long-term preference | **Limitation:** New listings need ~1 day of interaction data before appearing in results

GNN – Deep Dive Ch. 10 – PYMK / LinkedIn

GNN VARIANT	KEY IDEA	WHEN TO PREFER
GCN	Aggregates neighbor features via normalized adjacency	Static, homogeneous graphs
GraphSAGE ✓	Samples fixed-size neighborhood; learns aggregation (mean/LSTM/pool)	Large-scale graphs; inductive (handles new nodes)
GAT	Attention weights on neighbors	Heterogeneous neighbor importance
GIN	Sum-based aggregation; maximally expressive (WL-test equivalent)	When structural differences matter

PYMK Training: Input = social graph snapshot at time t + node features (age, account age, #connections) + edge features (mutual connections, profile visits). Task = predict which edges form at t+1 (edge-level prediction).

Why GNN over pointwise LTR here? Pointwise LTR ignores social context. FoF (Friends of Friends) signal – the #1 predictive feature for PYMK – is intrinsically a multi-hop graph property that GNNs capture natively.

Ad Click / Ranking Model Progression – Full Stack Ch. 8

MODEL	ARCHITECTURE	PROS	CONS	USE WHEN
LR	Linear combination → sigmoid	Fast, interpretable, baseline	Linear only	Always as baseline
Feature Cross + LR	Manually create feature pairs → LR	Captures pairwise interactions	Manual; sparse	Domain knowledge available

MODEL	ARCHITECTURE	PROS	CONS	USE WHEN
GBDT / XGBoost	Sequential decision trees (boosting)	Easy data prep; interpretable; great on structured	X No continual learning	Static data; explainability needed
GBDT + LR	GBDT extracts features → LR	Better features	Slow; no continual learning	Historical ad data
DCN (Deep Cross Network)	Deep DNN + Cross network (auto feature crosses) → concat → output	Auto feature crosses; no manual engineering	Cross network limited in interaction order	Industry standard
FM (Factorization Machine)	$\hat{y} = w_0 + \sum w_i x_i + \sum_{i \neq j} (v_i \cdot v_j) x_i x_j$ — pairwise via embedding dot products	Handles sparse high-cardinality features	2nd-order only	Sparse categorical features
DeepFM ✓	FM (low-order) + DNN (high-order) sharing same input embedding → combined output	Both low + high-order; no manual feature engineering	Slower than pure FM	Best overall for CTR prediction
GBDT + DeepFM	GBDT extracts features → DeepFM	Competition-winning	X Slow; slows continual learning	Competitions; offline batch

Multi-task DNN – Deep Dive Ch. 5 Harmful Content • Ch. 10 News Feed

Architecture: [Fused Features] → Shared Layers (DNN hidden layers) → Transformed Features → Task-Specific Classification Heads (1 per task)

- › **Shared layers:** Common feature transformation beneficial to all tasks. Data-sparse tasks borrow signal from data-rich tasks. Prevents redundant computation.
- › **Task-specific heads:** Independent DNN layers per task; each specializes for one prediction; separate loss per head.

SYSTEM	TASKS	LOSS
News Feed Meta	Like, Share, Comment, Click, Reaction + Dwell-time (regression) + Skip (binary)	Weighted sum: binary CE per classification + MAE/MSE for dwell-time
Harmful Content	Violence, Nudity, Hate speech (separate head per harm class)	Binary CE per harm class; combined weighted loss

Multi-task vs N independent DNNs: Single model to maintain; data-sparse tasks benefit from shared learning; prevents redundant feature transforms.

Challenges: Loss weight tuning is hard; one modality can dominate training → address with **gradient blending** or **focal loss**.

PART C – SPECIALIZED Domain-specific architectures for object detection, vision search & tree models

Object Detection Models Ch. 3 – Street View Blurring

ARCHITECTURE	HOW IT WORKS	SPEED	ACCURACY	USE WHEN
Two-Stage R-CNN, Faster R-CNN	Stage 1: RPN scans conv feature map → candidate regions. Stage 2: classifier assigns class per region	Slow (sequential)	High ✓	Moderate data (~1M); offline batch; accuracy > speed
One-Stage YOLO, SSD	Single network generates bounding boxes + class labels simultaneously; no explicit region proposals	Fast ✓	Lower	Real-time detection; latency-sensitive; very large datasets
DETR Transformer-based	End-to-end transformer attention; no NMS needed	Moderate	High ✓	Emerging best practice; clean pipeline

Training Loss: $L = L_{cls} + \lambda \cdot L_{reg}$ | L_{cls} = cross-entropy (classify box content) | L_{reg} = MSE over (x, y, w, h) bounding box coordinates

NMS (Non-Maximum Suppression): Keeps highest-confidence box, removes overlapping ones. IOU threshold determines true positives. **Frequently asked — must know.**

Interview rule: Start with Faster R-CNN for moderate data. Switch to YOLO when dataset grows or real-time needed.

Representation Learning & Contrastive Models Ch. 2 – Visual Search

ARCHITECTURE	PICK WHEN	KEY TRAIT
ResNet (CNN)	Moderate data (~1M images)	Local features; translation invariant
ViT (Vision Transformer)	Large data (100M+)	Global context; data-hungry

POSITIVE SAMPLE METHOD	PRO	CON
Human annotation	Accurate	Expensive, slow
User clicks as proxy	Fast, scalable	Noisy, sparse
Self-supervision (SimCLR, MoCo) ✓	Free, automated	Augmented ≠ real similarity

Contrastive Loss Pipeline: query_img → embed(E_q) | n images → embed($E_1..E_n$) → dot-product similarities → Softmax → Cross-entropy vs. ground truth index

Similarity measures: Dot product ✓ (preferred) | Cosine similarity | Euclidean (avoid in high dims — curse of dimensionality) | **ANN libraries:** Faiss (Meta), ScaNN (Google)

GBDT: Bagging vs. Boosting – Full Comparison Ch. 7 – Event Rec

TECHNIQUE	HOW	REDUCES	SPEED	EXAMPLES
Bagging	Train N trees in parallel on data subsets; combine via voting	Variance (overfitting)	Fast (parallel)	Random Forest
Boosting ✓	Train weak classifiers sequentially; each focuses on mistakes of previous	Both bias and variance	Slower (sequential)	AdaBoost, XGBoost, GradientBoost
GBDT		Bias + variance	Sequential	XGBoost ✓

TECHNIQUE	HOW	REDUCES	SPEED	EXAMPLES
	Gradient descent in function space; each tree fits residuals of previous			

Interview rule: If the interviewer asks about continual/online learning → immediately flag GBDT as a bottleneck. Prefer NNs for systems that retrain daily (Ad Click, News Feed, Event Rec).

All 10 Systems – Model Architecture Summary

SYSTEM	CANDIDATE GEN	RANKING MODEL	ARCHITECTURE TYPE	COLD START
Visual Search	ANN on embeddings	ResNet/ViT + Contrastive (SimCLR)	Representation Learning	✓ Image features
Street View Blurring	RPN	CNN Classifier (Faster R-CNN)	Object Detection	N/A (offline batch)
YouTube Search	ANN on embeddings	Two-Tower (text ↔ video encoder)	Two-Tower NN	✓ Video features
Harmful Content	Upload trigger	Multi-task DNN + Early Fusion	Multi-modal Multi-task	N/A (classification)
Video Rec	MF + Two-Tower ANN	Two-Tower NN (content features)	Hybrid CF + Content-based	Two-Tower uses demographics
Event Rec	Geo + category filter	XGBoost → DNN (Pointwise LTR)	Ranking (Pointwise LTR)	Online learning pipeline
Ad Click Prediction	Advertiser targeting	DeepFM	CTR Prediction	✓ Ad features available
Similar Listings	ANN on embeddings	Session-based W2V + neg. sampling	Session-based Embedding	Hard negatives; 1-day delay
News Feed	Friends + interests filter	Multi-task DNN (7 tasks)	Pointwise LTR + Multi-task	Demographic-based scoring
PYMK	FoF + shared attributes	GNN / Two-Tower edge prediction	Graph Neural Network	Partial: profile features help

Step 5 · Evaluation

Mental model → metric selection → 10-system reference → calibration → A/B rigor → fairness → retrieval eval → interview ritual

The 3-Stage Evaluation Pipeline

Always evaluate each stage independently

Stage 1 · Retrieval Eval

Recall@k vs exact brute-force · Hit-rate@k · MRR at retrieval

"Is the relevant item even in the candidate set?"



Stage 2 · Ranking Eval (Offline)

nDCG · mAP · MRR · PR-AUC · ROC-AUC · NCE · Avg rank of booked listing

"Are items ordered correctly on held-out data?"



Stage 3 · Online Eval (A/B Test)

CTR · Watch time · Conversion · Revenue lift · Acceptance rate · Prevalence

"Did the model move the North Star business metric?"

The Gap that kills careers: A model that improves Stage 2 but not Stage 3 is a production failure. The offline metric became a Goodhart target. Always validate offline→online correlation before trusting any single proxy metric.

Metric Hierarchy: North Star → Guard → Proxy

Define all three before naming any metric

TYPE	DEFINITION	EXAMPLE (VIDEO REC)	FAILURE IF IGNORED
North Star	The single long-term business KPI the company ultimately optimises for. Never directly trained on.	7-day retention · Total weekly watch time	Optimising CTR → clickbait → retention drops despite short-term gains
Guard Metrics	Metrics that must NOT degrade even if the North Star improves. Hard constraints on the trade-off space.	Content diversity · p99 latency <200ms · Fairness across demographics	Model boosts watch time via filter bubble → diversity collapses → churn spike
Proxy Metric	What the model actually optimises. Should correlate with North Star — the gap between them is the trap.	CTR · Watch completion rate · Explicit likes	CTR +15% but 6-month retention drops — Goodhart's Law in action

Interview ritual: State all three before naming any metric. "My North Star is [X]. My guard metrics are [Y, Z]. My proxy that I'll optimise is [W] — and here's why W correlates with X."

Offline Metric Decision Tree

Follow this order — don't jump straight to nDCG

Step 1

Output type?



Step 2

relevant items?



Step 3

Binary or graded?



Step 4

Classes balanced?



Step 5

Calibration needed?

STEP 1: OUTPUT TYPE	STEP 2–4: CONDITIONS	→ USE THIS METRIC	DO NOT USE
Classification	Balanced classes · Overall model quality	ROC-AUC	F1 alone (threshold-dependent)
			ROC-AUC — optimistic on imbalance

STEP 1: OUTPUT TYPE	STEP 2–4: CONDITIONS	→ USE THIS METRIC	DO NOT USE
	Imbalanced classes (harm <1%, fraud)	PR-AUC ← prefer over ROC-AUC	
	Probability calibration critical (ad auction)	Log-loss + NCE + ECE	ROC-AUC alone — doesn't measure calibration
Ranking	One relevant item per query	MRR	nDCG — requires graded labels you don't have
	Multiple relevant items · Binary labels	mAP	nDCG — needs non-binary scores
	Multiple relevant items · Graded labels (0–5)	nDCG	mAP — throws away the graded signal
Object Detection	Bounding box + class label quality	mAP @ IOU (0.5 / 0.75)	Accuracy — ignores box localisation
Embedding / Retrieval	ANN quality vs exact search	Recall@k vs brute-force · Avg rank of booked	nDCG — doesn't measure retrieval approximation gap
Regression	Continuous output (watch time, duration)	Huber Loss / MAE	MSE alone — outliers dominate
NLP Generation	Text overlap with reference (summarisation, translation)	BLEU · ROUGE · METEOR	Not for tasks where multiple valid answers exist; poor semantic sensitivity

Ranking Metrics Deep Dive

METRIC	FORMULA / INTUITION	USE WHEN	DO NOT USE WHEN	BOOK SYSTEM
Precision@k	relevant in top-k / k	Binary relevance · Quick sanity check	Rank order within top-k matters	Video Rec (secondary)
Recall@k	relevant in top-k / total relevant	Missing items is costly · Retrieval stage eval	Large total-relevant set → misleading denominator	Retrieval (all systems)
MRR Mean Reciprocal Rank	mean(1 / rank of 1st relevant)	Exactly one relevant item per query	Multiple relevant items exist — ignores everything after rank 1	YouTube Video Search
mAP Mean Avg Precision	mean AP over queries	Multiple relevant items · Binary relevance	Graded relevance — binary labels waste signal	Event Rec · PYMK · Video Rec
nDCG Normalised DCG	DCG/IDCG; $DCG = \sum rel_i / \log_2(i+1)$	Graded relevance (0–5 editorial labels)	Binary click data only — use mAP instead	Visual Search · Video Rec · News Feed
IOU Intersection over Union	$ A \cap B / A \cup B $ pred box vs ground-truth	Object detection box quality. IOU ≥ 0.5 = TP. Threshold choice (0.5/0.75/0.95) affects AP calc.	Not standalone — combine with AP to form mAP@IOU	Street View Blurring

Book usage quick-ref: [mAP](#) → Event Rec, PYMK, Video Rec (binary: registered/connected) · [nDCG](#) → Visual Search, News Feed (graded 0–5) · [MRR](#) → YouTube Video Search (one ground-truth video per query)



FAANG INTERVIEW PROBE

“Why mAP for Event Rec and not nDCG?”

Event registration is **binary** — registered (1) or not (0). There are no graded scores like 0–5.
nDCG requires non-binary labels to be meaningful; here it wastes signal that doesn’t exist.
mAP handles multiple binary-relevant items and captures rank position through Average Precision.

Ranking Metrics – Formula Reference

NEW

PRECISION@K

$$\text{Precision@}K = \frac{|\{\text{relevant items in top-}K\}|}{K}$$

K = cut-off rank · **numerator** = count of relevant items in top-K positions

❗ Does not penalise rank position within the window — quick sanity check only.

RECALL@K

$$\text{Recall@}K = \frac{|\{\text{relevant items in top-}K\}|}{|\{\text{all relevant items in catalog}\}|}$$

denominator = total relevant items in full catalog (not just retrieved)

❗ Primary metric for Stage 1 ANN retrieval. Low Recall@K = good items invisible to ranker.

AVERAGE PRECISION (AP) & MAP

STEP 1 — AP FOR ONE QUERY

$$\text{AP} = \frac{1}{|R|} \sum_{k=1}^K \text{Precision@}k \cdot \mathbb{1}[\text{item at rank } k \text{ is relevant}]$$

STEP 2 — MEAN OVER ALL QUERIES

$$\text{MAP} = \frac{1}{Q} \sum_{q=1}^Q \text{AP}_q$$

|R| = total relevant in catalog · **$\mathbb{1}[\cdot]$** = indicator (1 if relevant, else 0) · **Q** = number of queries

❗ Rewards early hits. Divides by |R| (not just retrieved) — penalises missed relevant items. Used: Event Rec, PYMK, Street View (mAP@IoU).

MEAN RECIPROCAL RANK (MRR)

$$\text{MRR} = \frac{1}{Q} \sum_{q=1}^Q \frac{1}{\text{rank}_q}$$

rank_q = position of the first relevant item for query q · **Q** = number of queries

❗ Only cares about the first hit — use only when exactly one canonical correct answer exists. Used: YouTube Video Search.

nDCG — NORMALISED DISCOUNTED CUMULATIVE GAIN

STEP 1 — DCG@K

$$\text{DCG@}K = \sum_{k=1}^K \frac{2^{\text{rel}_k - 1}}{\log_2(k+1)}$$

STEP 2 — IDEAL DCG (PERFECT RANKING)

$$\text{IDCG@}K = \sum_{k=1}^K \frac{2^{\text{rel}^*_k - 1}}{\log_2(k+1)}$$

STEP 3 — NORMALISE

$$\text{nDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}$$

rel_k = graded relevance at rank k (e.g. 0–5) · rel^*_k = ideal grade at rank k · $\log_2(k+1)$ = position discount

① Use when relevance is graded (0–5), not binary. Subsumes MAP for binary labels. Used: Visual Search, Video Rec, News Feed.

Metric selection rule — state these two facts before naming any metric:

1. **Relevance type:** binary (click / no-click) → MAP or MRR · graded (0–5 editorial) → nDCG
2. **Answer count per query:** one correct answer → MRR · multiple relevant items → MAP / nDCG

PR-AUC vs ROC-AUC — When Each Wins

DIMENSION	ROC-AUC	PR-AUC
Sensitive to imbalance?	Optimistic — TN dominates	Focuses on positive class only
Positives rare (<1%)	✗ misleading	✓ harm, fraud, rare events
Balanced classes	✓ overall quality	Less discriminating
Axes	TPR vs FPR	Precision vs Recall
Systems	News Feed · Ad Click	Harmful Content · Fraud

Trap: A classifier predicting "not harmful" for all 500M posts achieves ROC-AUC ≈ 0.98 because TN rate is huge. PR-AUC would be near zero — the real signal.

Calibration — Critical for Ad Auctions

Why it matters: In ad auctions, $P(\text{click})$ directly sets bid prices. If model outputs 0.8 but true CTR is 0.4, bids are systematically wrong → revenue loss with zero alert.

CONCEPT	DETAIL
ECE Expected Calibration Error	Bins predictions; measures avg gap between predicted prob and empirical accuracy per bin. ECE = 0 is perfect. Track alongside AUC.
Reliability Diagram	Plot predicted prob (x) vs actual accuracy (y). Perfect = diagonal. Overconfident = below diagonal.
Platt Scaling	Post-hoc sigmoid fit on held-out calibration set. Fast; works for well-behaved scores.
Isotonic Regression	Non-parametric, more flexible. Use when Platt residuals are non-monotonic.
NCE	$\text{CE}(\text{model}) / \text{CE}(\text{background CTR})$. NCE < 1 = beats naive baseline.

Rule: Recalibrate after every significant retrain. Large NNs are systematically overconfident — softmax ≠ true probabilities.

All 10 Systems — Offline + Online Metrics Unified Reference

SYSTEM	KEY OFFLINE METRIC	WHY THAT METRIC	PRIMARY ONLINE METRICS	⚠ WATCH-OUT
Visual Search Pinterest	nDCG (graded 0–5)	Relevance graded by human editors; mAP	CTR · Daily time spent	

SYSTEM	KEY OFFLINE METRIC	WHY THAT METRIC	PRIMARY ONLINE METRICS	⚠ WATCH-OUT
	+ Recall@k (ANN)	wastes graded signal. Recall@k separately evaluates ANN retrieval quality.		Graded labels are expensive — if only click data, fall back to mAP
Street View Blurring Google	mAP @ IOU $\geq$ 0.5 + FNR per skin-tone group	Object detection standard. FNR per Fitzpatrick scale = legal fairness hard wall. Recall > Precision is a legal constraint.	User reports & complaints	Batch system — no CTR. Fairness slice by Fitzpatrick scale is mandatory, not optional.
YouTube Video Search	MRR + Recall@k (retrieval)	One canonical best video per query $\rightarrow$ MRR. Decouple ANN retrieval quality from ranker separately.	Completion rate $\cdot$ Watch time $\cdot$ CTR	Completion rate > CTR for quality. CTR alone rewards clickbait titles.
Harmful Content Meta	PR-AUC (per harm class) + ROC-AUC	Severe imbalance (<1% harmful). ROC-AUC optimistic; PR-AUC focuses on minority positive class. Both tracked.	Harmful impressions $\cdot$ Valid appeals $\cdot$ Proactive rate $\cdot$ Prevalence	Prevalence = post count. Harmful impressions = reach. Viral post (1M views) >> 100 posts (10 views).
Video Recommendation YouTube	nDCG + Precision@k + Diversity score	Multiple relevant videos with graded engagement. Diversity = avg pairwise cosine dissimilarity in top-k list.	CTR $\cdot$ # completed videos $\cdot$ Total watch time $\cdot$ Explicit feedback	CTR alone = clickbait. Must pair with completion rate and watch time together.
Event Recommendation Eventbrite	mAP	Binary relevance (registered/not). Multiple relevant events. MRR wrong (multiple). nDCG wrong (binary labels).	CTR $\cdot$ Conversion rate $\cdot$ Bookmark rate $\cdot$ Revenue lift	Conversion is lagged. Define attribution window before measuring.
Ad Click Prediction	Log-loss (CE) + NCE + ROC-AUC + ECE	Calibration-critical: P(click) sets bid prices. NCE vs naive baseline. ECE measures calibration quality independently.	CTR $\cdot$ Conversion rate $\cdot$ Revenue lift $\cdot$ Hide rate	High CTR + high hide rate = irrelevant ads. Hide rate is the false-positive alarm for ad quality.
Similar Listings Airbnb	Avg rank of booked listing + Recall@k (ANN)	Ground truth = which listing was booked in session. Lower avg rank = better. Ties offline metric directly to booking intent.	CTR $\cdot$ Session book rate	Session book rate is delayed. Use CTR as proxy but confirm book rate before calling A/B winner.
News Feed Facebook	ROC-AUC per task + nDCG (engagement score) + MAE (dwell-time)	Multi-task DNN: one AUC per head (like, share, click). Dwell-time is regression $\rightarrow$ MAE. Final feed ranked by engagement score $\rightarrow$ nDCG.	CTR $\cdot$ Reaction rates $\cdot$ Total time spent $\cdot$ User satisfaction survey	Passive users invisible to reaction metrics. Dwell-time added specifically for users who scroll but never react.
	mAP			

SYSTEM	KEY OFFLINE METRIC	WHY THAT METRIC	PRIMARY ONLINE METRICS	⚠ WATCH-OUT
PYMK LinkedIn		Multiple relevant connections. Binary (connected/not). Temporal split required — i.i.d. split is invalid for graph edges.	# requests sent · # requests accepted	Sent ≠ Accepted. Optimising "sent" recommends influencers nobody hears back from. Acceptance rate is the true North Star.

Online Evaluation Rigor — Running A/B Tests Correctly

CONCEPT	WHAT IT IS	SILENT FAILURE IF IGNORED
MDE Min Detectable Effect	Smallest effect worth detecting; drives required sample size. Compute before launching: $n \approx 2\sigma^2(z\alpha + z\beta)^2 / \text{MDE}^2$	Under-powered test → false negative. Real improvement missed.
No peeking	Pre-commit to sample size and end date. Never repeatedly check p-value mid-experiment.	Repeated peeking inflates FP rate → ship a regression.
Novelty effect	New models get temporary CTR boost from curiosity. Run ≥ 2 weeks before calling a winner.	Great week-1, stagnation or regression in week 3.
SUTVA violation Network effects	In social systems (PYMK, News Feed), user A's rec affects user B. Individual-level split is biased. Fix: cluster-level or geo-based split.	Control contaminated. Effect estimate wrong.
Interleaving	Mix old + new model items in same list. User implicitly compares both. More sensitive with less traffic than A/B.	Not suitable when item exposure itself is the treatment.

🔍 FAANG PROBE — A/B TESTING

“How long do you run an A/B test?”

Until two conditions are both met: **(1)** The MDE-required sample size is reached — pre-compute this before launching using $n \approx 2\sigma^2(z\alpha + z\beta)^2 / \text{MDE}^2$. **(2)** The novelty

Offline → Online Gap: IPS & Backtesting

CONCEPT	DETAIL	WHEN TO USE
Backtesting	Replay historical data through new model; compare decisions vs old model before any live traffic.	Ad systems · Fraud · High-stakes decisions
IPS Inverse Propensity Scoring	Reweight each sample by $1/P(\text{item was shown})$. Without IPS, offline eval always favours the incumbent model.	Offline eval on logged data · Debiasing training labels
IPS limitation	High variance when propensity is very small (tail items). Use clipped IPS or doubly robust estimators.	Large catalogs with near-zero propensity items
Goodhart's Law	"When a measure becomes a target, it ceases to be a good measure." Validate offline→online correlation across iterations.	Always — never ship on a single offline metric gain

🔍 FAANG PROBE — BACKTESTING

“How do you evaluate a new ranking model before A/B testing?”

Run **IPS-corrected backtesting** on historical logged data (reweight each sample by $1/P(\text{item was shown})$), then run **shadow deployment** to validate score distributions match expectations — before any live traffic is split.

“You said optimise for CTR — how do you handle clickbait?”

Track **completion rate**, **hide rate**, and **conversion rate** alongside CTR. Use re-ranking to demote high-CTR low-

effect has decayed — minimum **2 weeks** for recommendation systems. Calling a winner at day 3 is p-hacking; calling it at day 60 wastes traffic.

quality content: Final Score = $w_1 \cdot \text{CTR} + w_2 \cdot \text{completion} - w_3 \cdot \text{hide_rate}$.

STAFF-LEVEL PROBE

“Your nDCG improved 2% offline but CTR didn’t move in A/B. Why?”

Root cause: Offline eval uses historical logged data — items the old model showed are over-represented. Without IPS correction, the 2% gain is an artifact of selection bias, not real improvement.

Also check: Novelty effect — run ≥ 2 weeks

A/B power — was MDE pre-computed?

Wrong proxy — is CTR the right metric or should it be completion rate?

Fairness Evaluation – Street View, Harmful Content, Ad Click

METRIC	DEFINITION	EXAMPLE
Demographic Parity	$P(\hat{y}=1 \text{group A}) = P(\hat{y}=1 \text{group B})$ — equal positive prediction rate across groups	Street View: equal blur rate across Fitzpatrick skin tones
Equalized Odds	Equal TPR and FPR across groups — not just prediction rates	Harmful Content: equal recall + FP rate across age/ gender groups
Calibration by Subgroup	$P(y=1 \hat{y}=p, \text{group}) = p$ for all groups — don't check only aggregate ECE	Ad CTR: bid prices must be calibrated per demographic, not globally
Eval Breakdown Rule	Always slice by age, gender, skin tone, geography. Never report aggregate only.	mAP=0.92 global; 0.71 dark skin tones → GDPR liability

Staff signal: Raise fairness proactively without being asked. “I’ll slice eval by [demographic]. Aggregate metrics hide subgroup failures — in Street View that’s a GDPR hard wall.”

Stage 1 Retrieval Evaluation – Decouple from Ranker

METRIC	WHAT IT MEASURES	USE WHEN
Recall@k for ANN	How often is the true nearest neighbor returned by Faiss/ ScaNN vs exact brute-force? Measures approximation loss.	Every system with ANN — Visual Search, YT Search, Video Rec, Similar Listings, PYMK
Hit-rate@k	Fraction of queries where ≥ 1 relevant item appears in top-k before ranking stage.	Candidate gen eval. Low hit-rate = ranker has nothing to work with.
MRR at retrieval	MRR on retrieval output alone, before ranking. Decouples retrieval from ranker quality.	Systems with one dominant relevant item. Debug: is failure in retrieval or ranking?
t-SNE / UMAP	Qualitative sanity check — similar items cluster, anomalies visible.	Debugging embedding quality after first Two-Tower or contrastive model training.

Principle: A ranker on bad candidates can never recover. If Recall@100 = 0.60, 40% of good items are forever invisible — fix retrieval first, then the ranker.

Evaluation Interview Ritual – Say This, In This Order

#	WHAT TO SAY	WHY IT SIGNALS SENIORITY
1	"My North Star is [X]. My guard metrics are [Y, Z] — must never degrade. My proxy is [W], which I'll use for offline eval."	Juniors jump to metrics. Seniors anchor in business hierarchy first.
2	"I'll evaluate in 3 stages: retrieval quality, offline ranking quality, then online A/B. Let me describe each."	Shows production system thinking — not just "train and measure AUC".
3	"For offline ranking I choose [mAP/nDCG/MRR] because [relevance is binary/graded, one/multiple relevant items]. I would NOT use [X] because [specific reason]."	The "would NOT use" sentence is the differentiator. Shows metric understanding, not memorisation.
4	For harm/ad click: "Severe class imbalance → PR-AUC over ROC-AUC. For ad click I also track ECE because P(click) sets bid prices directly."	Calibration awareness = production awareness. Most candidates never mention ECE.
5	"For online eval I'd run A/B. Pre-compute sample size from MDE, run ≥2 weeks for novelty decay, cluster-level splits for social systems to avoid SUTVA violations."	Proactive SUTVA mention for social systems is a Staff-level differentiator.
6	"Before going live, I'd backtest using IPS-corrected offline replay — naive eval on logged data is biased toward the incumbent model."	IPS/backtesting separates engineers who've shipped from those who haven't.
7	"Finally I'd slice all eval metrics by [relevant demographic]. Aggregates hide subgroup failures — in Street View that's GDPR liability, not an edge case."	Proactive fairness without being asked signals ownership and production readiness.

🚫 MOST COMMON EVALUATION MISTAKE

Naming a metric without justifying why it fits this specific task.

✗ SCORES 2/5

"I'd use nDCG."

✓ SCORES 5/5

"I'd use nDCG **because** relevance is graded 0–5 — human editors score images — so I need a metric that rewards placing a score-5 image above score-3, not just relevant vs irrelevant. I would **NOT** use mAP because it treats relevance as binary and discards the graded signal."

Step 6 · Deployment & Serving

Serving: Batch/Online decision · Safe release · Train-serve skew · Retraining pipeline · Model Store · Edge deployment

PART A — SERVING ARCHITECTURE

Decide batch vs. online first — everything else flows from this

Batch vs. Online Prediction

DIMENSION	BATCH PREDICTION	ONLINE PREDICTION
When to use	Items known in advance; results can be pre-computed	Query is unknown; results depend on real-time input
Latency	No serving latency (pre-computed)	Latency = model inference time
Freshness	Stale (computed periodically)	Fresh (real-time)
Use cases	PYMK (batch weekly), Street View Blurring, Similar Listings	Ad Click, Event Rec, Video Search, News Feed
Risk	Wasted compute for inactive users	Model may be too slow; needs optimization

ANN (Approximate Nearest Neighbor) for Embedding Retrieval

ANN TYPE	ALGORITHM	PICK WHEN
Exact NN	Linear scan $O(N \cdot D)$	Small N ($<1M$); must guarantee accuracy
Tree-based	R-tree, KD-tree, Annoy	Moderate scale; low dimensionality
LSH-based	Locality-Sensitive Hashing	High dimensionality; hash similar points to same bucket
Clustering-based ✓	FAISS (Meta), ScaNN (Google)	Billion-scale systems; best industry choice

Book uses ANN for: Visual Search (billions of images), YouTube Video Search, Similar Listings (5M listings), Video Recommendation

PART B — SAFE RELEASE

How to ship without blowing up production

Testing in Production — Strategy Decision

STRATEGY	HOW	TRADE-OFF
Shadow Deployment	New model runs in parallel; only old model serves	Safe; costly ($2 \times$ compute)
A/B Testing	Split traffic randomly; measure metric difference	Requires sufficient traffic for statistical significance
Canary Release	Gradually route traffic $1\% \rightarrow 10\% \rightarrow 100\%$	Limits blast radius; slower rollout
Interleaving Experiments	Mix items from old + new model in same list	More sensitive; less traffic needed
Multi-armed Bandits	Dynamically allocate traffic to better-performing model	Faster convergence; less wasted traffic

Train-Serving Skew – The #1 Production Killer

☆ STAFF+

First Principle (Chip Huyen): "The model is correct, but the feature list used is wrong – or the featurization code is outdated." The most common Staff-level failure mode.

SKEW TYPE	CAUSE	FIX
Feature skew	Training uses batch historical features; serving computes them differently	Feature Store with shared transformation code
Label skew	Training labels from a period that no longer matches serving distribution	Temporal train/eval splits + drift monitoring
Log-and-Wait skew	Features logged at prediction time differ from features recomputed at training time	Log & reuse features from the prediction service directly

The Fix: A **Feature Store** (Feast, Tecton, Vertex AI Feature Store) is the canonical answer. It enforces the same transformation code at training time and serving time – eliminating all three skew types.

🔗 **FAANG Probe:** "Your model performs well in offline eval but degrades in the first week of production. Root-cause it."

→ Train-serving skew: feature pipeline divergence between training and inference. Fix with a Feature Store that shares the same transformation logic across both paths.

PART C – RETRAINING PIPELINE

4 stages: manual → scheduled → stateful → continual

4 Stages of Continual Learning (Chip Huyen Ch.9)

☆ STAFF+

HOW TO SELF-DERIVE CONTINUAL LEARNING NEED – 2 Questions (No need to be told)

- › **Q1:** "How fast does the data change?" → **data velocity**
- › **Q2:** "What is the penalty for a stale prediction?" → **staleness cost**

🧠 Mental rule: **Staleness penalty × Data velocity = Retraining urgency**. Infer from the system prompt – if the interviewer says "ad bidding" or "trending content," you already know the answer.

DATA VELOCITY	STALENESS COST	DECISION
Minute-to-minute (ad auctions, bidding)	Direct revenue loss per stale bid	Continual – stateful, delta fine-tune. GBDT eliminated.
Daily (trending topics, news, events)	Engagement drops within days	Daily retrain – stateful or stateless per data size
Weekly / monthly (social graph, listings)	Slow drift; users won't notice short-term	Scheduled stateless – GBDT valid
Near-static (road images, medical scans)	No measurable drift	Monthly / quarterly

STAGE	DESCRIPTION	BLOCKER TO NEXT STAGE
	Ad hoc retrain from scratch; triggered by engineer. Painful, slow.	Lack of automation

STAGE	DESCRIPTION	BLOCKER TO NEXT STAGE
Stage 1 Manual Stateless		
Stage 2 Automated Scheduled	Cron-driven full retrain (e.g., nightly). Still stateless — full data each cycle.	High compute cost; no triggers
Stage 3 Automated Stateful	Checkpoint-based retraining on delta data only. Requires model lineage tracking & Model Store.	No streaming pipeline; no drift triggers
Stage 4 Trigger-Based Continual	Drift-triggered retraining via streaming pipeline. System is fully self-healing.	—

Staff Answer Template: "We're currently at Stage 2. To reach Stage 4, we need: (1) a streaming feature pipeline (Kafka + Flink), (2) PSI-based drift triggers, and (3) a Model Store with lineage tracking."

🔗 **FAANG Probe:** "How would you evolve your retraining infrastructure from weekly batch to real-time continual learning?"

→ Walk through Stage 2→4 progression: Stage 2=cron retrain; Stage 3=checkpoint+delta data (needs Model Store); Stage 4=drift-triggered streaming (needs Kafka/Flink + PSI monitors). Name the blocker at each stage.

Stateless vs. Stateful Training

☆ STAFF+

DIMENSION	STATELESS RETRAINING	STATEFUL TRAINING
What	Full retrain from scratch each cycle	Continue training from last checkpoint
Data	All historical data each run	Only delta (new) data each run
Cost	High (e.g., 100M rows daily)	Low (only incremental data)
Risk	Stable, reproducible, no lineage needed	Model lineage tracking required; harder to debug
Model compatibility	Works with GBDT, DNN, any model	Neural networks only — GBDT cannot be fine-tuned incrementally
Use when	Weekly/monthly cadence; stable distribution	Hourly/continuous retraining; high data velocity

GBDT Trap: XGBoost/LightGBM cannot be incrementally updated — each new tree depends on global residuals. For systems needing daily/continuous retraining, you must use Neural Networks (DeepFM, Two-Tower) to enable stateful training.

🔗 **FAANG Probe:** "Your ad click model needs daily retraining on 100M new rows. Stateless or stateful?"

→ Stateful with NN (DeepFM): continue from last checkpoint on only the new delta data. Stateless would require reprocessing the full corpus daily — too costly. GBDT is disqualified because it cannot be incrementally fine-tuned.

Model Store – 8 Required Artifacts

☆ STAFF+

A Model Store (MLflow, Vertex AI Model Registry, SageMaker Registry) must track all 8 artifacts to enable full reproducibility and debugging without the original engineer.

#	ARTIFACT	WHY CRITICAL
1	Model definition	Architecture code (layer config, hyperparams)
2	Model parameters	Saved weights/checkpoint
3	Featurize + predict functions	Same code used at training AND serving — prevents skew
4	Dependencies (container image)	Library versions; reproducible environment
5	Data pointer (DVC version)	Exact dataset version used for this model
6	Experiment metadata	Hyperparams, training config, run ID
7	Tags / owners	Oncall contact, team, model purpose
8	Evaluation results	Offline metrics at time of registration; rollback baseline

🕒 **FAANG Probe:** "Your production model degraded and the responsible data scientist is on leave. How do you debug it?"

→ Model Store with full lineage: reproduce via stored artifacts, compare feature schema versions between current and last-good model, roll back to the last registered model with passing eval results.

PART D – EDGE / SPECIALIZED

On-device, compression — only if Requirements flagged it

Cloud vs. On-Device

DIMENSION	CLOUD	ON-DEVICE
Simplicity	✓ Easy	X Complex deployment
Cost	X Cloud cost	✓ No cloud cost
Network latency	X Present	✓ None
Inference speed	✓ Faster (GPU)	X Slower
Privacy	X Data leaves device	✓ Data stays local
Internet dependency	X Required	✓ Not needed

Model Compression Techniques

TECHNIQUE	HOW	USE WHEN
Knowledge Distillation	Train small student to mimic large teacher	Large model needed offline; small needed for serving
Pruning	Zero out least useful parameters → sparse model	Reduce storage; post-training
Quantization	32-bit → 8-bit representation	Reduce model size and inference latency

Step 7 · Monitoring & Infrastructure

Why systems fail · What to monitor · Shift detection · Retrain triggers · Frequency by system · Feedback loops

PART A — WHAT & WHY TO MONITOR ————— Anchor monitoring in business failure modes before adding alerts

Why Systems Fail in Production

ROOT CAUSE	DESCRIPTION	DETECTION
Data distribution shift	Training data no longer represents serving data (e.g., seasonal trends)	Monitor input feature distributions
Upstream data change	Data pipeline schema changes; feature values shift	Monitor feature statistics
Model staleness	World changes; model hasn't adapted	Track offline metric degradation on recent data
Software bugs	Feature computation errors at serving time	Shadow testing; integration tests

What to Monitor

CATEGORY	METRICS
Operational	Serving latency, QPS, CPU/GPU utilization
Input / Output	Feature value distributions, output score distributions
Drift Detection	Statistical tests on input feature drift, label drift
Model Accuracy	Offline metrics on recent window vs. threshold
Model Version	Which model is live; rollback capability

PART B — DRIFT DETECTION ————— Types of shift → statistical tests → when to trigger retraining

3 Types of Data Distribution Shift (Chip Huyen Ch.8)

☆ STAFF+

First Principle: "If you don't do anything, model performance always degrades over time. The question is only how fast."
— Chip Huyen. Models don't expire on a calendar; they expire when the world changes.

SHIFT TYPE	FORMAL DEFINITION	DETECTION METHOD
Covariate Shift	$P(X)$ changes; $P(Y X)$ stays the same — input distribution drifts but relationship is intact	KS test / PSI on feature distributions
Label Shift	$P(Y)$ changes; $P(X Y)$ stays the same — output prior shifts (e.g., fraud prevalence increases)	Monitor output score distributions, label prevalence
Concept Drift	$P(Y X)$ changes — the relationship itself changes; the world changed around the model	Monitor accuracy on recent labeled windows — feature tests won't catch this

Staff Instinct: For any domain shift question (seasonality, new user cohort, regulatory change), proactively classify as covariate/label/concept drift and name your detection mechanism. Never say just "distribution shift" — name which type.

🔗 **FAANG Probe:** "User behavior shifted dramatically post-COVID. Which type of drift is this and how do you detect it?"

→ Primarily concept drift: $P(Y|X)$ changed — users now click on different content types than before (the relationship changed). Detect via accuracy degradation on recent labeled data windows, NOT just feature distribution tests. Covariate shift tests would not catch this.

Statistical Drift Tests by Feature Type

☆ STAFF+

TEST	FEATURE TYPE	THRESHOLD / NOTES
KS Test (Kolmogorov-Smirnov)	Continuous features (age, price, duration)	$p < 0.05$ = significant drift; sensitive but high false-positive rate
PSI (Population Stability Index)	Any feature; industry standard for monitoring	$PSI < 0.1$ = stable; $0.1-0.2$ = moderate; > 0.2 = major shift → retrain
Chi-squared test	Categorical features (country, device, category)	Tests if observed vs. expected category frequency changed
Two-sample t-test	Mean shift detection in continuous features	Simpler than KS; only detects mean shift, not distributional change
MMD (Maximum Mean Discrepancy)	High-dimensional embedding drift	Best for detecting drift in embedding spaces (Two-Tower, BERT)

Staff Template: "For continuous features like user_age, I use KS test with PSI threshold 0.2. For categorical features like country, chi-squared. For embedding space drift in our Two-Tower model, MMD."

🔗 **FAANG Probe:** "How do you detect input drift in production for your recommendation system?"

→ Per feature type: KS test + $PSI > 0.2$ for continuous features (age, session length); chi-squared for categorical (country, device type); MMD for Two-Tower embedding drift. Alert on $PSI > 0.2$ on top-20 features by SHAP importance.

PART C – ACTING ON DRIFT ————— Triggers → frequency by system → special failure modes

Retraining Triggers – 4 Types

☆ STAFF+

Staff Instinct: Always propose a retraining trigger, not just a frequency. Frequency implies a calendar — you retrain whether or not drift occurred. Trigger implies your system is responsive. The latter signals Staff-level production maturity.

TRIGGER TYPE	WHEN TO USE	RISK
Time-based every N hours/days	Stable, predictable data velocity; simplest to implement	May retrain unnecessarily when no drift occurred; wastes compute
Performance-based accuracy < threshold	Need labeling pipeline to compute accuracy in near real-time	Labels are often delayed (fraud: 30 days) — not always feasible

TRIGGER TYPE	WHEN TO USE	RISK
Volume-based N new labeled samples	Reliable when natural labels flow fast (CTR, clicks)	Ignores distribution change if volume is constant but distribution shifted
Drift-based ✓ PSI > 0.2 on key features	Best for production; directly measures the actual problem	False positives if monitoring too many features; tune carefully

Best Practice: Combine **volume-based + drift-based** triggers: retrain when 5% new labeled data accumulated OR PSI > 0.2 on top features, whichever comes first. Add a data freshness experiment before committing: train on month-old vs. week-old data — the performance delta justifies the infra investment.

🔗 **FAANG Probe:** "How do you decide when to retrain your ad click prediction model?"
→ Volume + drift-based combined trigger: retrain when 5% new labeled impressions accumulated OR PSI > 0.2 on top SHAP features, whichever comes first. This avoids wasteful time-based retraining while being responsive to actual distribution change.

Retraining Frequency Decision

FREQUENCY	USE WHEN	SYSTEMS
Continuous / near-real-time	Data changes every minute; stale model loses revenue	Ad Click Prediction
Daily	Data evolves daily; user behavior changes fast	News Feed, Event Rec
Weekly	Slower-moving graph; lower data velocity	PYMK, Similar Listings
Monthly / on-demand	Mostly static data; low distribution shift	Visual Search, Street View Blurring

FAANG probe: "GBDT can't be fine-tuned — how do you handle continual training?" → Use Neural Networks which support incremental fine-tuning; avoid GBDT for systems requiring frequent retraining.

Degenerate Feedback Loop – Detection & Fix

☆ STAFF+

Definition: The model's own recommendations influence the training labels, causing the model to reinforce its own biases over time. A silent killer — accuracy metrics can look stable while catalog coverage collapses.

SIGNAL	WHAT IT TELLS YOU
Catalog coverage drop	Model serves only a small subset of available items — long-tail disappears from impressions
Item-rank distribution entropy ↓	Same items appear at rank 1 repeatedly; healthy distribution should have high entropy
CTR looks fine	Trap: the model gets better at recommending the same popular items. Users never see alternatives.
FIX	MECHANISM
ε-greedy exploration	With probability ε, serve a random item instead of the top-ranked one

FIX	MECHANISM
Thompson Sampling	Sample from posterior — naturally balances explore/exploit
Contextual bandits	Structured exploration per user context
Guard metric: catalog coverage	Set a hard floor — e.g., top-1000 recommendations must cover $\geq 10K$ unique items daily

🔗 **FAANG Probe:** "After 6 months in production, your recommender only surfaces 200 items out of a 1M catalog. What happened and how do you fix it?"

→ Degenerate feedback loop: the model trained on its own popular recommendations, reinforcing the same items. Fix: add ϵ -greedy or Thompson sampling exploration, monitor catalog coverage metric daily, set a guard metric on item distribution entropy.

Adversarial Inputs – Detection & Defense

☆ STAFF+

Definition: Users or external actors **deliberately craft inputs** to exploit model blind spots — inflate metrics, bypass classifiers, or game ranking. Unlike random noise, adversarial inputs are intentionally optimized against your model's decision boundary.

ATTACK TYPE	SYSTEM	HOW IT WORKS	SILENT RISK IF MISSED
Click Fraud	Ad Click (Ch8)	Bots generate fake clicks to inflate CTR → corrupts training labels → model learns fraudulent signal as ground truth	Model bids higher on fraudulent inventory → advertiser revenue wasted → trust collapses
Content Bypass	Harmful Content (Ch5)	Leetspeak (h4tē), adversarial image patches, text split across posts — all designed to evade harm classifiers	Harmful content reaches users; model confidence stays high — no alert fires
Rank Manipulation	Similar Listings (Ch9) / PYMK (Ch11)	Hosts/sellers create synthetic co-click signals or fake mutual connections to boost their item/profile in embedding space	Legitimate items displaced; model embedding space corrupted over time
Label Poisoning	Any user-labeled system	Coordinated fake reports or fake positive interactions to flip a model's training signal on a target item	Model learns inverted ground truth for targeted class; degrades silently

3-LAYER DEFENSE STRATEGY

LAYER	MECHANISM	WHEN APPLIED
① Ingestion Guard	Rate-limit signals per user/IP; anomaly detection on feature distributions (z-score > 3 on click rate = flag); bot fingerprinting	Before data enters training pipeline
② Adversarial Training	Augment training data with known adversarial examples (perturbed inputs, bypass patterns); model learns invariance to perturbation	During model training
③ Serving-Time Anomaly Detection	Monitor input feature distributions at inference; flag requests where features deviate from training distribution (PSI > 0.2); reject or route to fallback	At inference / real-time serving

Adversarial vs. Distribution Shift — don't confuse them:

Distribution shift = world changed naturally (COVID → user behavior changed).

Adversarial inputs = actor is actively working against you — the shift is intentional and targeted.

Detection looks similar (feature drift), but root cause and fix differ entirely.

FAANG Probe: "Your ad CTR model shows a sudden 30% spike in CTR on a specific advertiser. Is this a good model or a problem?" → Adversarial signal: likely click fraud. Investigate click-rate z-score per IP cluster, device fingerprint entropy, conversion rate (fraud clicks don't convert). Add ingestion-level anomaly guard. Do NOT retrain on this data — you'd poison the model.

Cross-Cutting Design Patterns

Set the bar first, then study the patterns – Senior/Staff mindset + 10 cross-system patterns

Senior AI/ML Interview Expectations

What's actually expected: fluency in the 7-step framework, confident model-choice justification, awareness of production pitfalls. Not memorizing every system perfectly.

- › **Lead with requirements** — 3–4 targeted questions before any design. This is the biggest seniority signal.
- › **Name the objective precisely** — "I'm framing this as a pointwise LTR problem to maximize registration probability" earns credibility.
- › **Justify model choices with trade-offs** — "I chose DeepFM over GBDT because we need continual learning as trends shift hourly."
- › **Proactively mention pitfalls** — Drop positional bias, delayed feedback, or cold start unprompted to signal depth.
- › **Multi-stage funnel is always correct** — Any recommendation or ranking system: candidate gen → ranking → re-ranking. Never skip this.
- › **CTR alone is always wrong** — Always pair CTR with a quality signal (completion rate, conversion rate, hide rate). Interviewers probe this.
- › **Offline → Online metric gap** — Know why nDCG doesn't tell you about clickbait, and how A/B testing closes the loop.
- › **AI/ML differentiator** — Tie retraining strategy to model type: "If we go with NN instead of GBDT, we can support continual learning."

Maximum ROI: Master the 7-step framework cold + know the canonical model choice for each of the 10 systems. The cross-cutting patterns are your senior-level flex — drop 2–3 unprompted per interview.

10 Must-Know Patterns

PATTERN	DESCRIPTION	USED IN
Feature Store	Pre-computed static features stored offline; dynamic features computed at query time	Ad Click News Feed Event Rec
Hard Negative Mining	Add incorrectly predicted examples as explicit negatives to retrain — forces the model to learn harder boundaries	Street View Blurring Similar Listings
Positional Bias	Items shown higher get more clicks regardless of quality → biases training labels; correct by position-aware training or IPS weighting	Visual Search Similar Listings News Feed
Clickbait Prevention	Track conversion rate + completion rate alongside CTR; use re-ranking to demote low-quality high-CTR content	Video Rec Event Rec
Cold Start	New items/users have no interaction history; Matrix Factorization fails; Two-Tower handles using features; session-based uses recent clicks	Video Rec PYMK Similar Listings
Two-sided Marketplace Fairness	Optimizing for users may disadvantage hosts/creators; need constraints or multi-objective optimization to balance both sides	Event Rec Similar Listings
Data Leakage	Features computed after the label event leak future info into training; always ensure temporal ordering in train/eval splits	All ranking / rec systems

PATTERN	DESCRIPTION	USED IN
Delayed Feedback	User books 3 days after clicking → labeling before 3 days creates premature negatives; use a feedback window or probabilistic labeling	Ad Click PYMK Similar Listings
Diversity in Re-ranking	Remove near-duplicates; ensure topic/style variety in top-k; use MMR (Maximal Marginal Relevance) or explicit dedup logic	Video Rec PYMK Visual Search
Continual Learning Bottleneck	GBDT cannot be incrementally fine-tuned — must retrain from scratch; NNs support continual learning; avoid GBDT for high-frequency retrain systems	Ad Click News Feed

⑩ Debugging Production Inference Failures

The D.I.A.L. Framework · Chip Huyen Ch.8 & Ch.10 · Silent failure detection → root cause attribution

△ The Core Problem: ML Fails Silently

Chip Huyen's #1 insight: ML systems fail silently — the code compiles, the endpoint returns 200, the loss looked fine in training, but predictions are **quietly wrong**. A traditional software failure gives you a 404 or segfault; an ML failure gives you **confident wrong answers with no alert firing**.

TRADITIONAL SOFTWARE FAILURE

- › Returns 404 / 500
- › Throws a segfault
- › Empty response body
- › Timeout / OOM
- › ✓ Alert fires immediately

ML SYSTEM FAILURE (SILENT)

- › Returns HTTP 200
- › Loss was fine in training
- › Predictions shift quietly
- › Revenue drops, no page
- › X No alert fires without monitoring

The D.I.A.L. Framework – 4-Layer Debugging Path

Walk through these 4 layers **in order**, from the outermost shell inward. Each layer must be **ruled out before descending**.

LAYER	NAME	WHAT YOU CHECK	CHIP HUYEN SOURCE
D	Detect	Is there actually a failure? Operational vs. ML metric violation	Ch.8 – Operational vs. ML performance expectations
I	Isolate	Where is it? Software bug vs. ML-specific failure	Ch.8 – 60% of Google's 96 ML pipeline breaks were non-ML causes
A	Attribute	What caused it? Train-serving skew, drift, staleness, or edge case	Ch.8 – 4 ML-specific failure types
L	Localize	Which component? Data → Feature pipeline → Model → Serving code	Ch.10 – Model Store 8-artifact checklist

Layer D – Detect: Is There a Real Failure?

Two types of expectation violations — distinguish them before acting:

TYPE	EXAMPLES	HOW IT SURFACES
Operational (easy)	Timeout, OOM, 404, segfault, empty response	Pages you immediately
	Accuracy drifts, predictions shift,	

Layer I – Isolate: Software Bug or ML Failure?

Google data (Chip Huyen Ch.8): 60 of 96 ML pipeline breaks over 15 years were **non-ML causes** — distributed systems errors, orchestration mistakes, incorrect data joins. **Always rule out software first.**

TYPE	EXAMPLES	HOW IT SURFACES
ML Performance (silent)	proxy metrics decouple from north star	Requires monitoring to surface

DETECTION CHECKLIST

- › p99 latency spiking? → **Operational, not model quality**
- › Prediction distribution shifting (more zeros, extreme scores)? → **ML signal**
- › User feedback (hide rate, completion rate, downvotes) degrading while CTR holds? → **ML objective misalignment**
- › Accuracy on held-out slices dropped? → **Distribution shift**

🔴 **FAANG Probe:** Your model has 95% accuracy offline but revenue dropped post-deployment. What happened?
Offline metric diverged from business KPI. Likely distribution shift, or ML objective (CTR) misaligned with north star (revenue). Fix: add online A/B metrics tied to business KPIs.

SOFTWARE FAILURE CHECKLIST (RULE OUT FIRST)

- › **Wrong model binary deployed** — did you push the right version? Check model registry artifact
- › **Dependency version change** — did a library upgrade silently change behavior?
- › **Permission/read error** — is the serving system reading the correct feature source?
- › **Orchestration failure** — did a DAG step fail silently, causing stale features to be served?
- › **Hardware fault** — GPU/CPU behaving incorrectly under thermal load

If all software causes are ruled out → proceed to Layer A (ML-specific).

Layer A – Attribute: The 4 ML Root Causes (Chip Huyen Ch.8)

A1. TRAIN-SERVING SKEW (Most Common)

Model is correct, feature list is correct, but featurization code at **serving computes features differently from training**.

SKEW TYPE	CAUSE	FIX
Feature skew	Training batch features ≠ serving compute (Python→Java rewrite)	Feature Store (Feast/Tecton) shared transform code
Label skew	Training labels from period no longer representative	Temporal train/eval splits + drift monitoring
Log-and-Wait	Features logged at prediction time ≠	Log features from prediction

A2. DATA DISTRIBUTION SHIFT

The world changed; the model didn't. Three subtypes:

SHIFT TYPE	DEFINITION	DETECTION
Covariate shift	$P(X)$ changes, $P(Y X)$ stays same	PSI / KS test on features
Label shift	$P(Y)$ changes	Monitor prediction dist. over time
Concept drift	$P(Y X)$ itself changes	Accuracy degrades even with stable input/output dist.

Detection tools: KS test, PSI (PSI > 0.2 = major shift → retrain). Monitor

A3. MODEL STALENESS / DEGENERATE FEEDBACK LOOP

Model was fine at launch but degraded as the world evolved. A **degenerate feedback loop** is a special case: the model's own predictions influence future training data, amplifying its biases.

Example:
A4. EDGE CASES
Recommender shows that the model was never trained on a single edge case label, so it can't reveal its fragility in the entire pipeline.

SKEW TYPE	CAUSE	FIX
	features recomputed at training time	service; reuse at training

🔍 **Probe:** Model works offline, degrades first week in prod → Train-serving skew. Fix with Feature Store.

prediction distribution shift as a cheap proxy for input shift.

- Example: Never-seen user segment, new device type, unusual text encoding
- Detection: **Slice-based monitoring** — track performance per subgroup, not just

Layer L – Localize: Which Component? (Model Store 8-Artifact Checklist)

INFERENCE FAILURE DECISION TREE

```

Inference request fails/degrades
├── Raw Input → Is upstream data schema the same?
├── Feature Pipeline → Is featurization code identical to training?
│   └── (Check Feature Store / logged features)
├── Model Binary → Is the correct model version deployed?
│   └── (Check Model Registry – definition + parameters)
├── Serving Infra → Is pre/post-processing code correct?
│   └── (Check featurize/predict functions artifact)
└── Dependencies → Are library versions identical to training env?
    └── (Check container image artifact)
  
```

MODEL STORE – 8 ARTIFACTS TO CHECK

#	ARTIFACT	WHY CRITICAL
1	Model definition	Architecture code (layer config, hyperparams)
2	Model parameters	Saved weights/checkpoint
3	Featurize + predict functions	Same code used at training AND serving — prevents skew
4	Dependencies (container image)	Library versions; reproducible environment
5	Data pointer (DVC version)	Exact dataset version used for this model
6	Experiment metadata	Hyperparams, training config, run ID
7	Tags / owners	Oncall contact, team, model purpose
8	Evaluation results	Offline metrics at registration; rollback baseline

🔍 **FAANG Probe:** Production model degraded, the DS is on leave. How do you debug?

Pull Model Store record → check model definition, featurize/predict code, data version pointer, and container. Re-run same inputs locally. If local ≠ production → delta is in serving pipeline. If they match → root cause is data drift or upstream schema change.

Monitoring Toolbox for Debuggability (Observability)

Chip Huyen's definition: Observability = the ability to infer internal system state from runtime outputs **without** shipping new code.

LOGS

- › Record every event with unique trace IDs, timestamps, user IDs, input hashes
- › Use distributed tracing — a 20–30 hop request must be fully traceable
- › Tag logs with subgroup metadata (zip code, device, segment)

DASHBOARDS

- › Visualize feature distributions and prediction distributions
- › Latency percentiles — always track **p50 AND p99**, never just average
- › **Slice-based accuracy by user segment, geography, device**

ALERTS

- › PSI > 0.2 on key features → major distribution shift
- › Prediction distribution shift beyond threshold
- › p99 latency SLA breach
- › **Avoid alert fatigue — only alert on actionable, critical signals**

🔑 **Observability rule (Chip Huyen):** You should be able to query: "Show me all users for which model A returned wrong predictions over the last hour, grouped by zip code" — this requires logs tagged with **subgroup metadata**.

Failure Mode Quick Reference

SYMPTOM OBSERVED	MOST LIKELY ROOT CAUSE	FIRST ACTION
Sudden accuracy collapse day-of-deploy	Wrong model binary / dependency change	Check Model Store deployment artifact
Gradual degradation over weeks	Distribution shift / model staleness	Run PSI on features; compare prediction distribution
Works locally, fails in production	Train-serving skew	Compare featurization code training vs. serving
High CTR, low completion, revenue drops	ML objective ≠ north star	Check proxy vs. north star metric alignment
Model predicts all-one-class	Class imbalance + degenerate loop	Check prediction distribution logs, retraining data
Works for most users, fails one segment	Edge case / subgroup	Slice-based eval by user/item subgroup
OOM / timeout / 404	Software / infrastructure	Treat as SWE bug first, not ML bug

Complete Debugging Runbook – Interview Answer Template

When asked "How would you debug a production inference failure?" — structure your answer around **D.I.A.L.**

D

Detect — Confirm it's a real failure: operational metric (latency, error rate) or ML metric (prediction shift, accuracy drop, user feedback degradation)?

I

Isolate — Rule out software: wrong binary, dependency failure, orchestration bug, hardware fault?

A

Attribute — Identify ML root cause: train-serving skew → data distribution shift → model staleness/degenerate feedback loop → edge case?

L

Localize — Use Model Store to trace which artifact is the source: raw input schema → feature pipeline → model binary → serving code → dependency container



Staff Badge: Proactively mention that **60% of production ML failures are non-ML software bugs (Google data)** before diving into distribution shift. This signals you debug **methodically**, not just from the ML angle.